

Поиск пути в графе.

Задача о кратчайшем пути

Задача о кратчайшем пути — задача поиска самого короткого пути(цепи) между двумя точками (вершинами) на графе, в которой минимизируется сумма весов рёбер, составляющих путь.

Постановки задачи

- Задача о кратчайшем пути в заданный пункт назначения
- Задача о кратчайшем пути между заданной парой вершин
- Задача о кратчайшем пути между всеми парами вершин

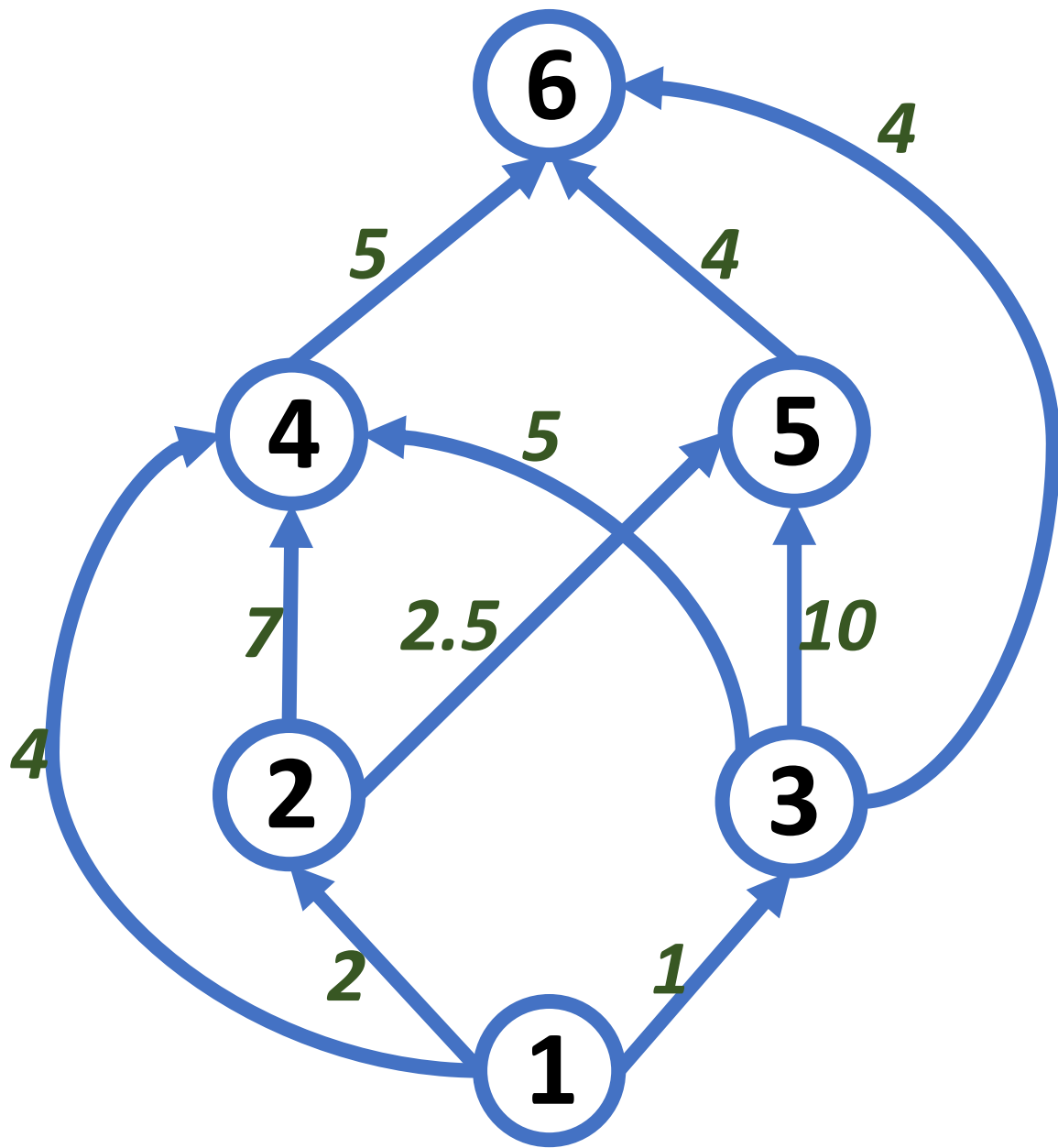
Алгоритм Дейкстры

- Находит кратчайший путь от одной из вершин графа до всех остальных. Алгоритм работает только для графов без ребер отрицательного веса.

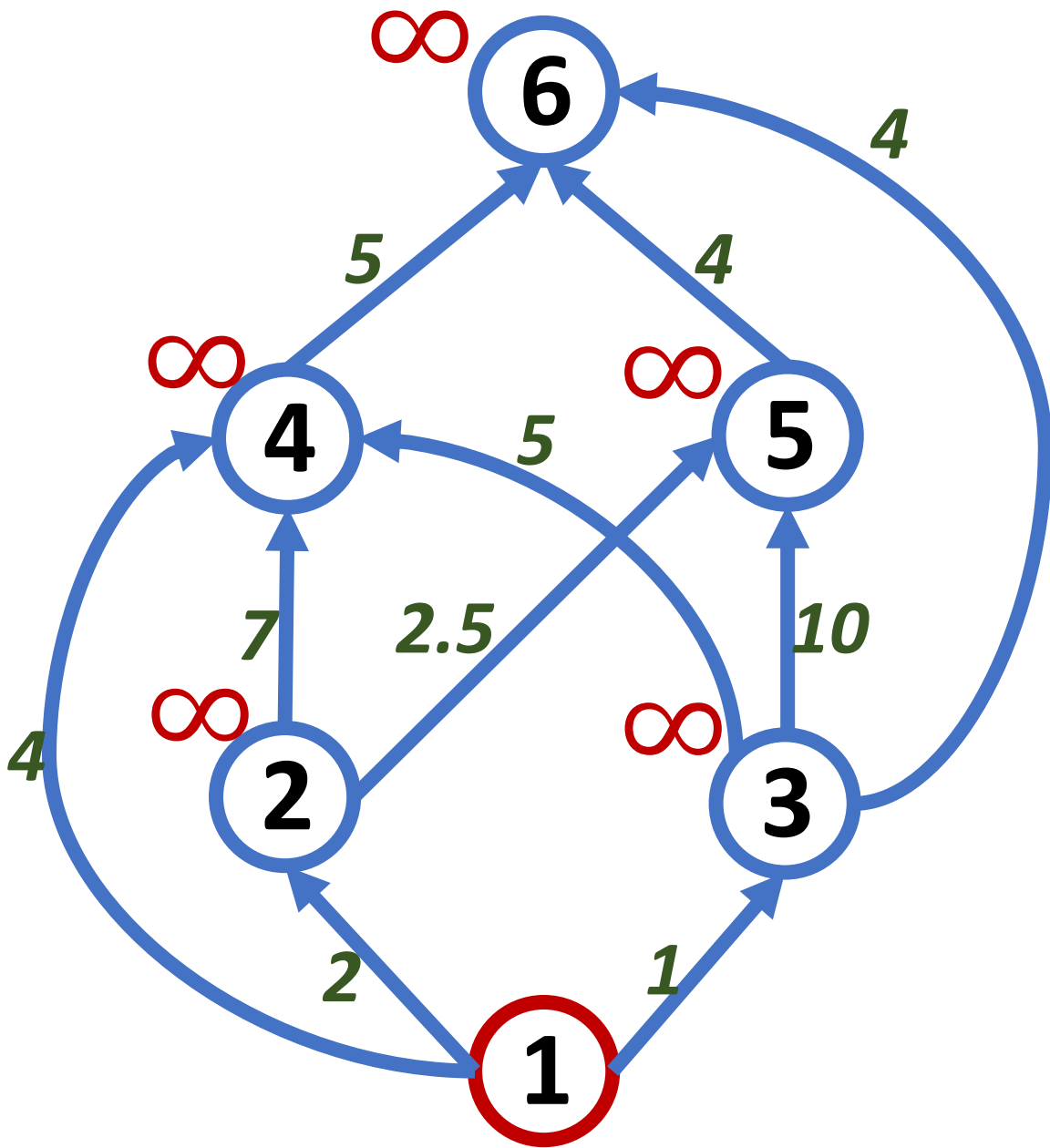
Алгоритм Дейкстры

- Шаг 1. Всем вершинам, за исключением первой, присваивается *вес* равный бесконечности, а первой вершине – 0.
- Шаг 2. Все вершины не выделены.
- Шаг 3. Первая *вершина* объявляется текущей.
- Шаг 4. *Вес* всех невыделенных вершин пересчитывается по формуле: *вес* невыделенной вершины есть минимальное число из старого *веса* данной вершины, суммы *веса* текущей вершины и *веса ребра*, соединяющего текущую вершину с невыделенной.
- Шаг 5. Среди невыделенных вершин ищется *вершина* с минимальным *весом*. Если таковая не найдена, то есть *вес* всех вершин равен бесконечности, то *маршрут* не существует. Следовательно, *выход*. Иначе, текущей становится найденная *вершина*. Она же выделяется.
- Шаг 6. Если текущей вершиной оказывается конечная, то *путь* найден, и его *вес* есть *вес* конечной вершины.
- Шаг 7. Переход на шаг 4.

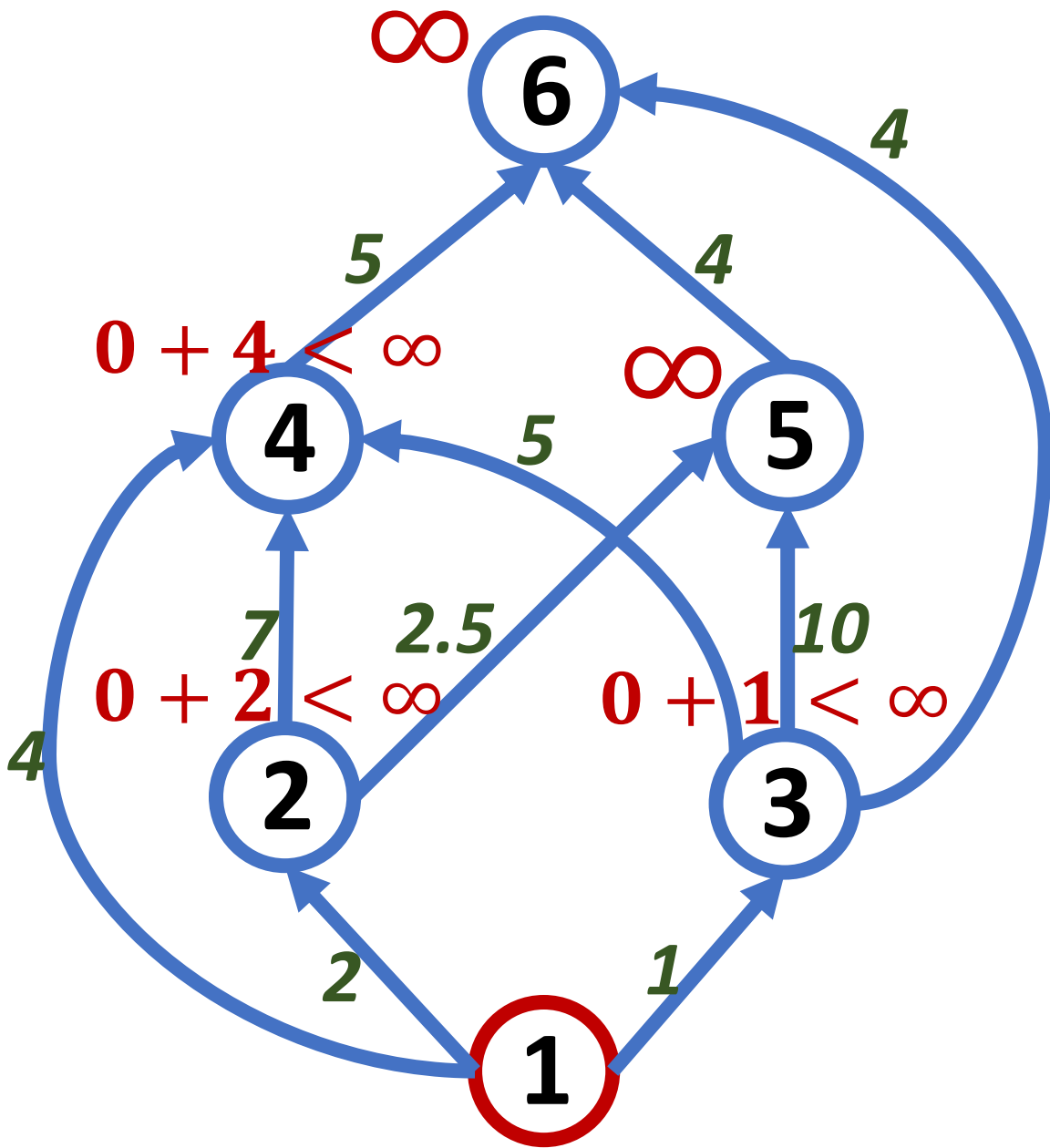
Найти кратчайший путь из вершины 1 в вершину 6



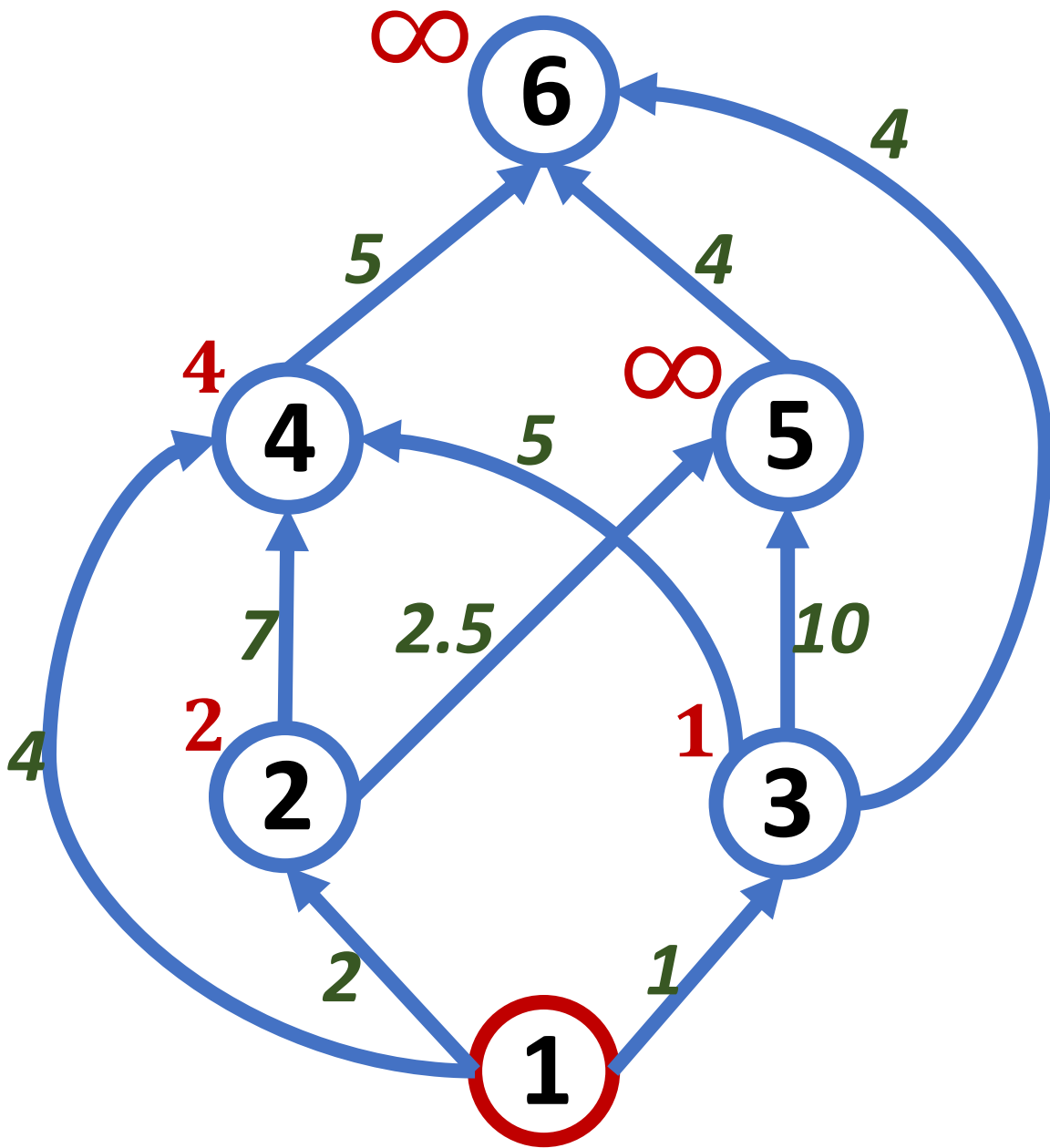
1	2	3	4	5	6
0	0	0	0	0	0



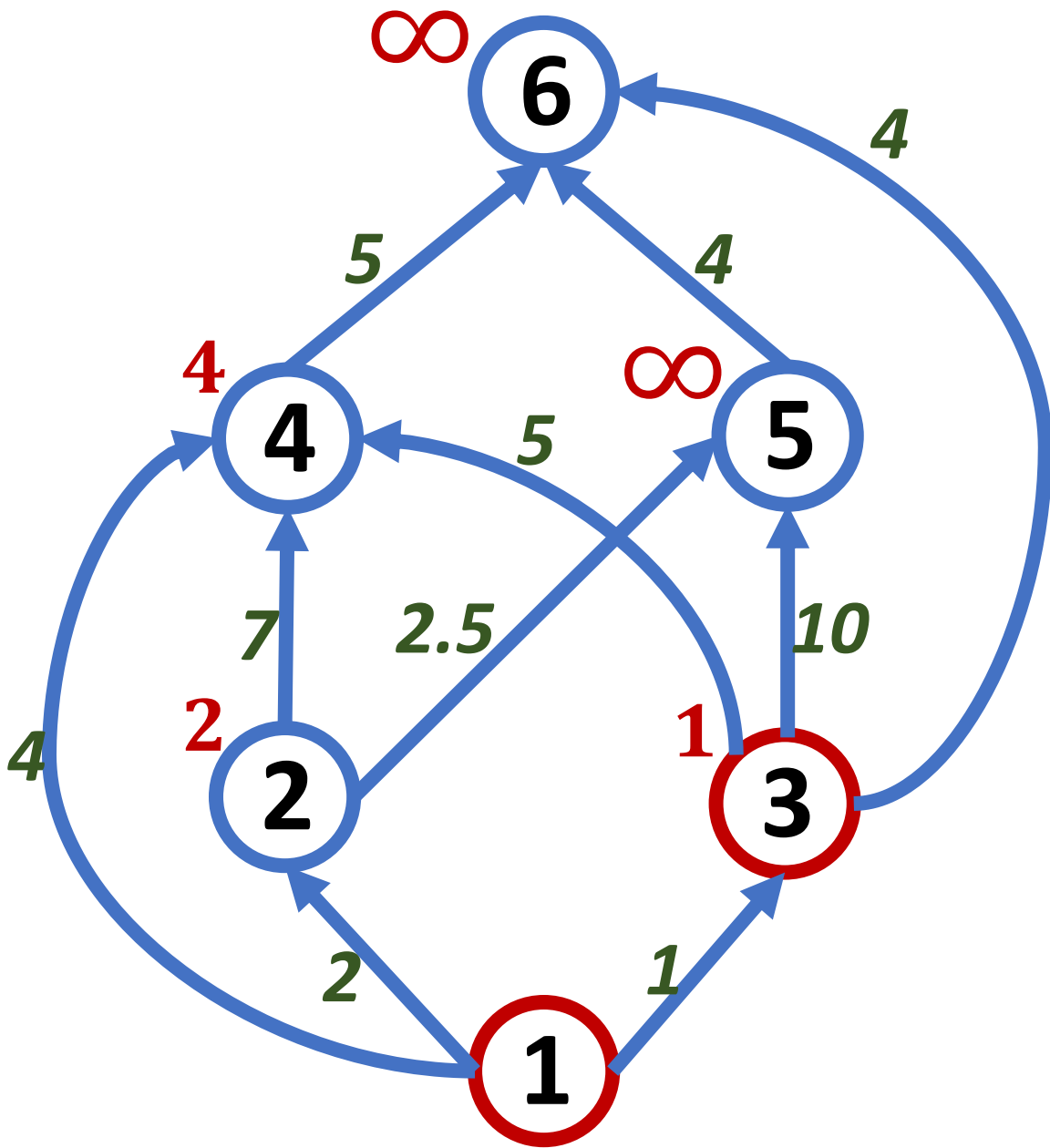
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
0	0	0	0	0	0



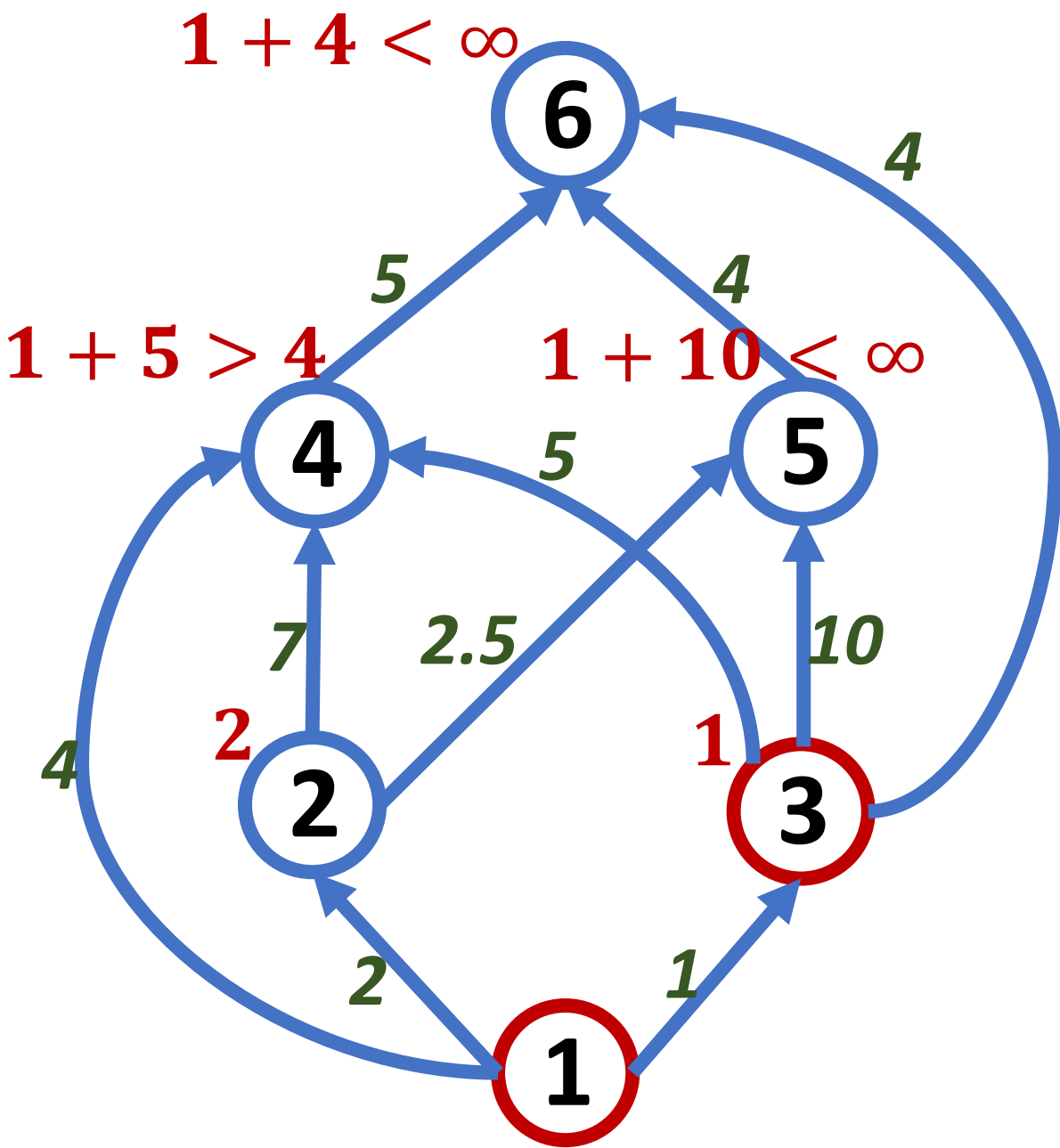
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	$0+2<\infty$	$0+1<\infty$	$0+4<\infty$		
0	0	0	0	0	0



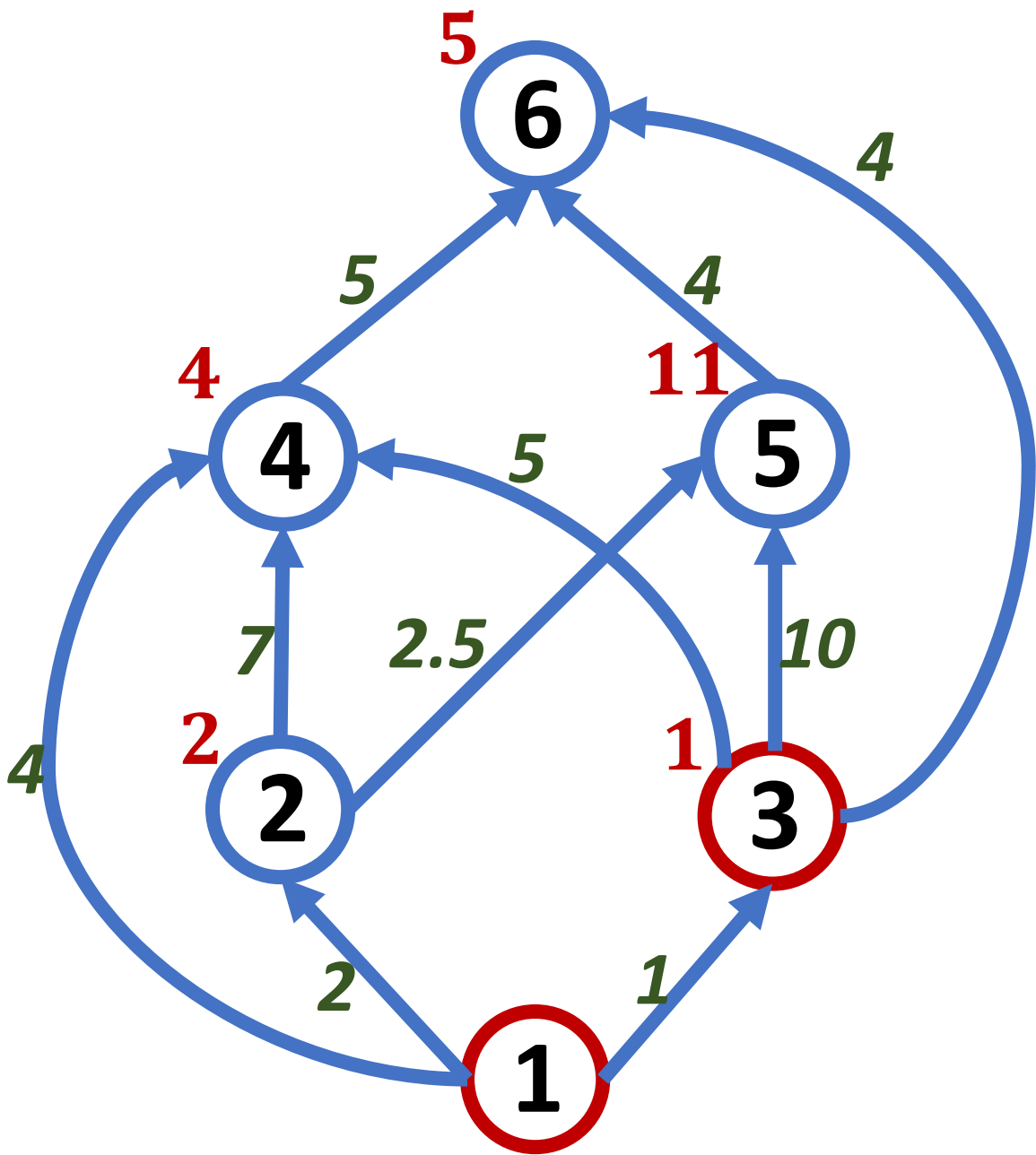
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
0	1	1	1	0	0



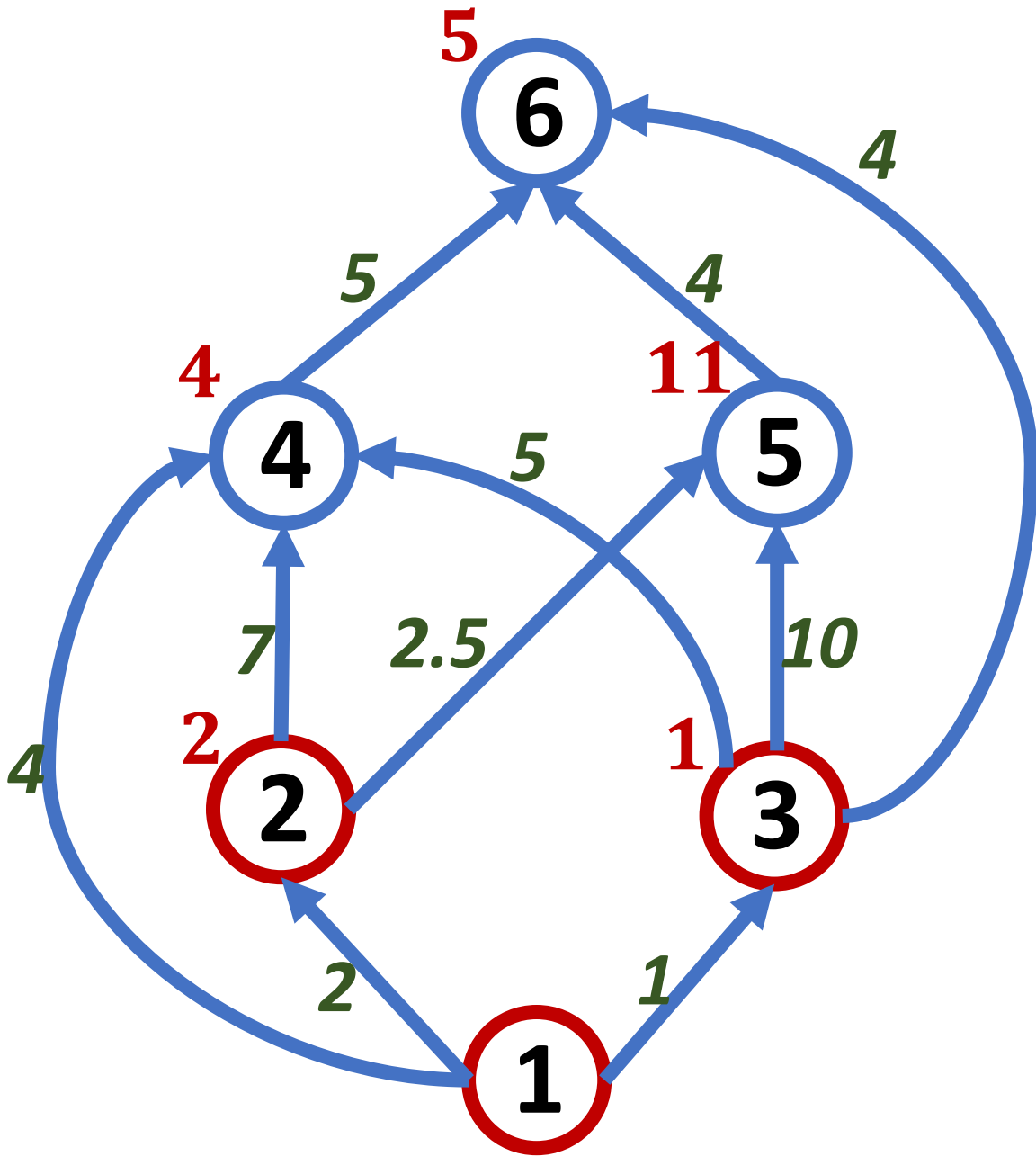
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
0	1	1	1	0	0



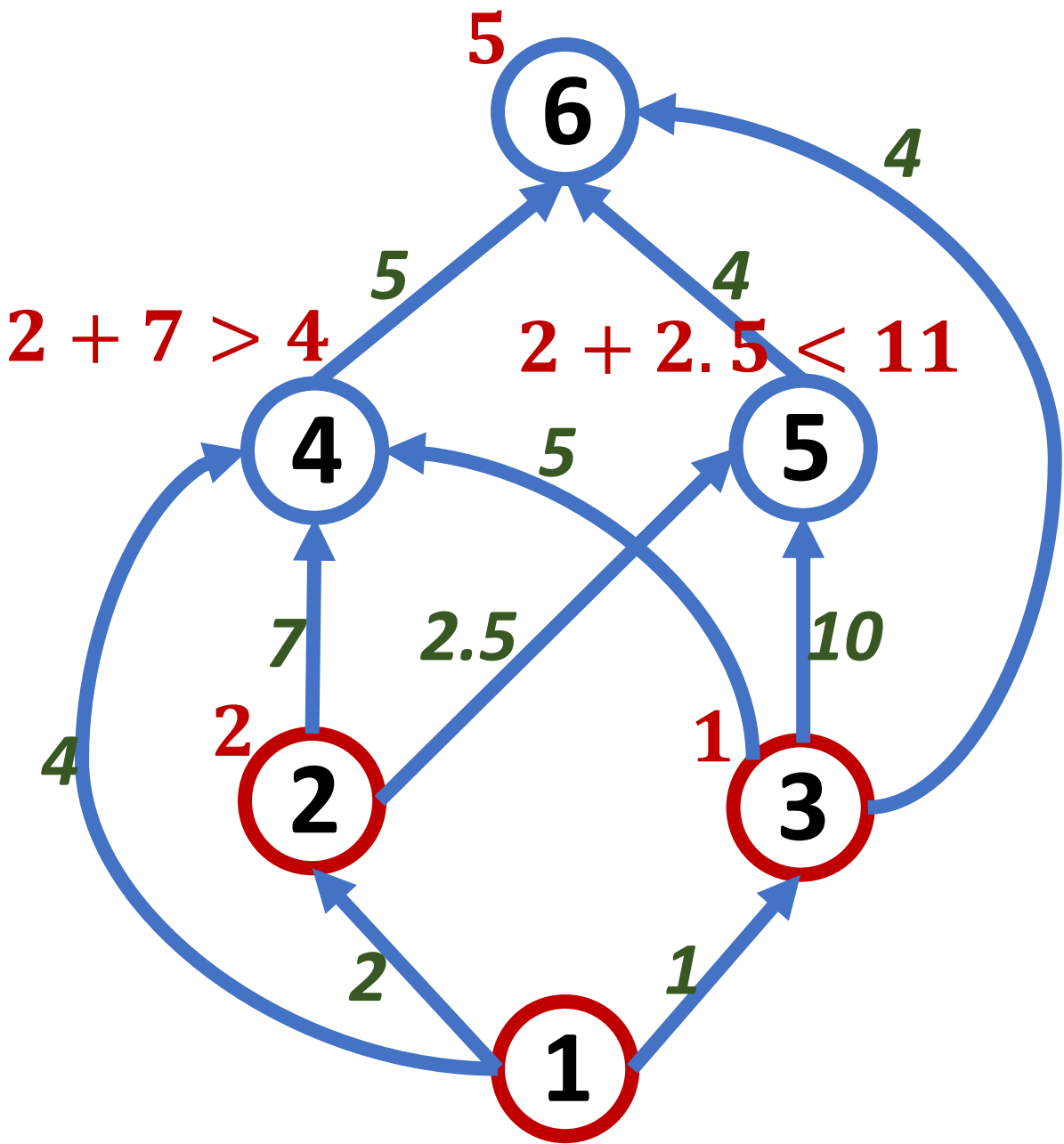
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		$1+5>4$	$1+10<\infty$	$1+4<\infty$
0	1	1	1	0	0



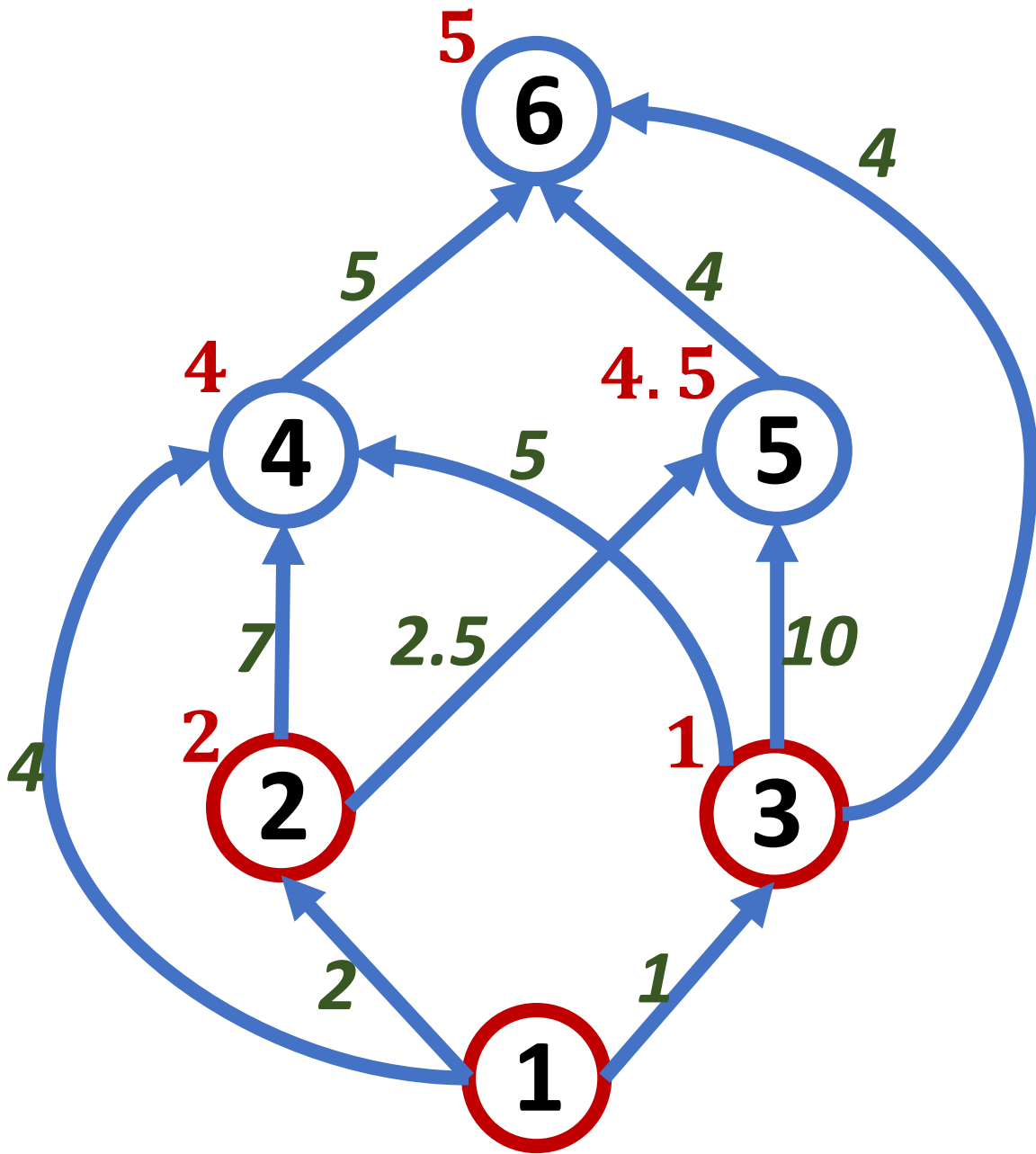
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
0	1	1	1	3	3



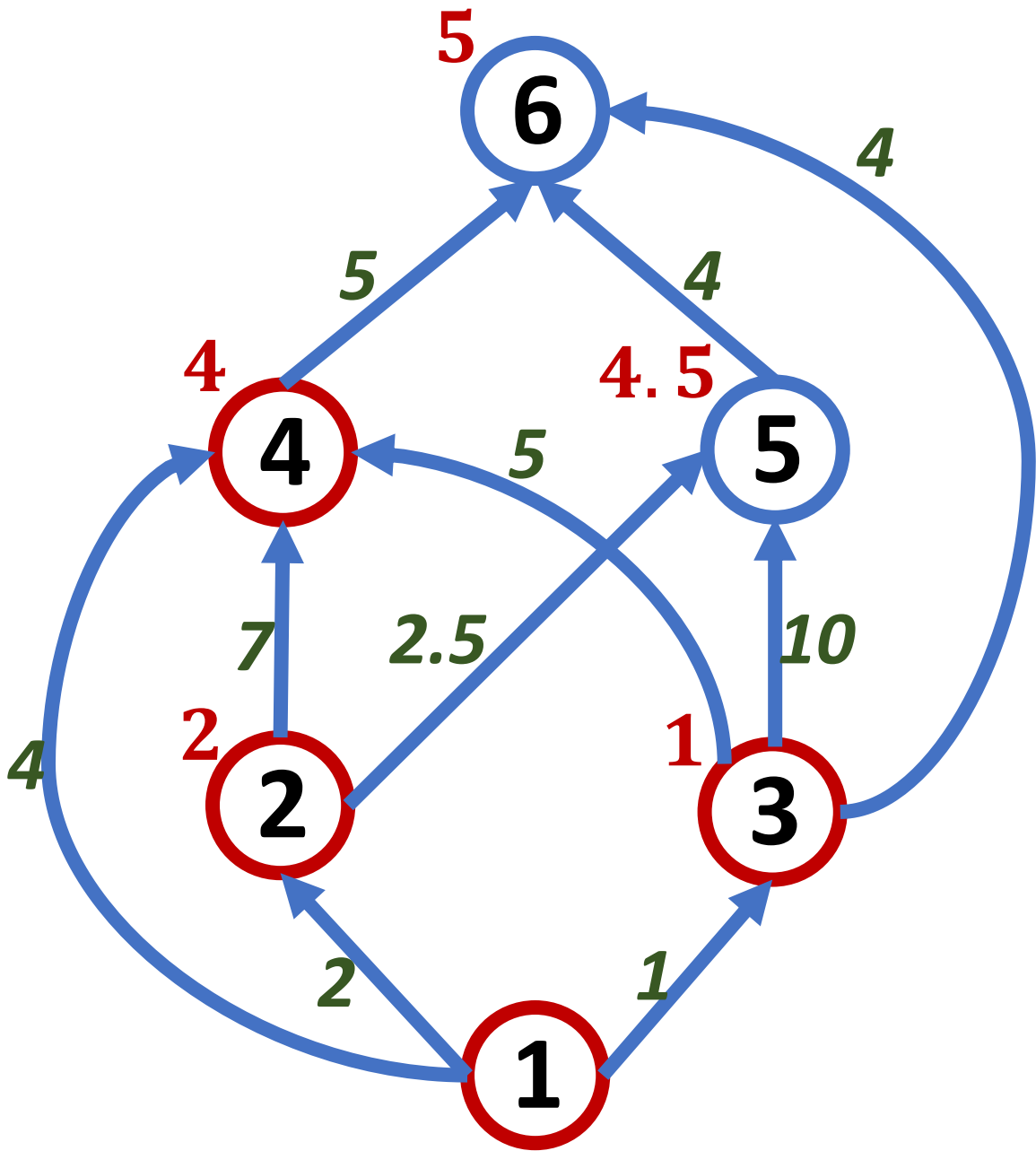
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
0	1	1	1	3	3



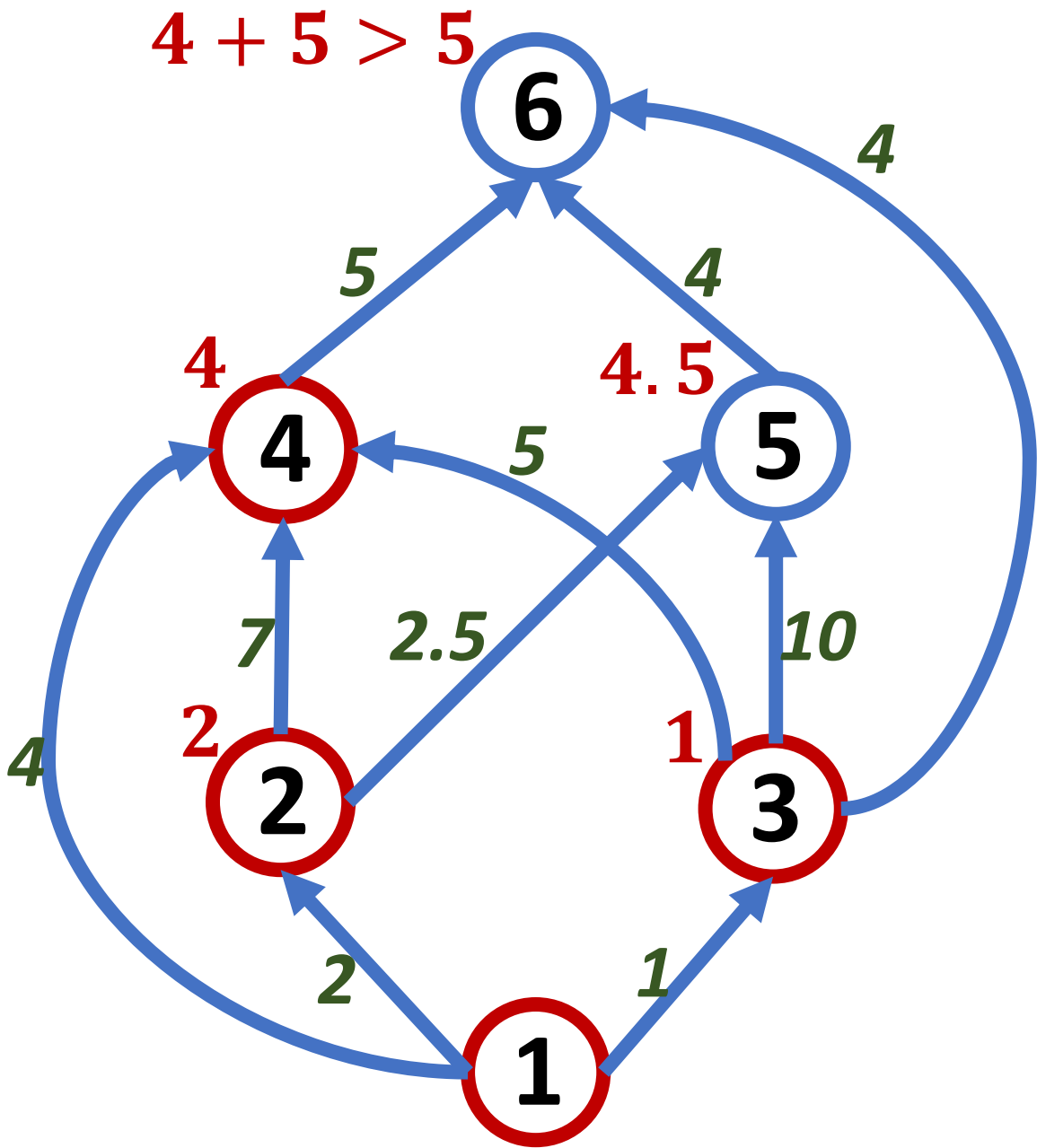
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			$2+7>4$	$2+2.5<11$	5
0	1	1	1	3	3



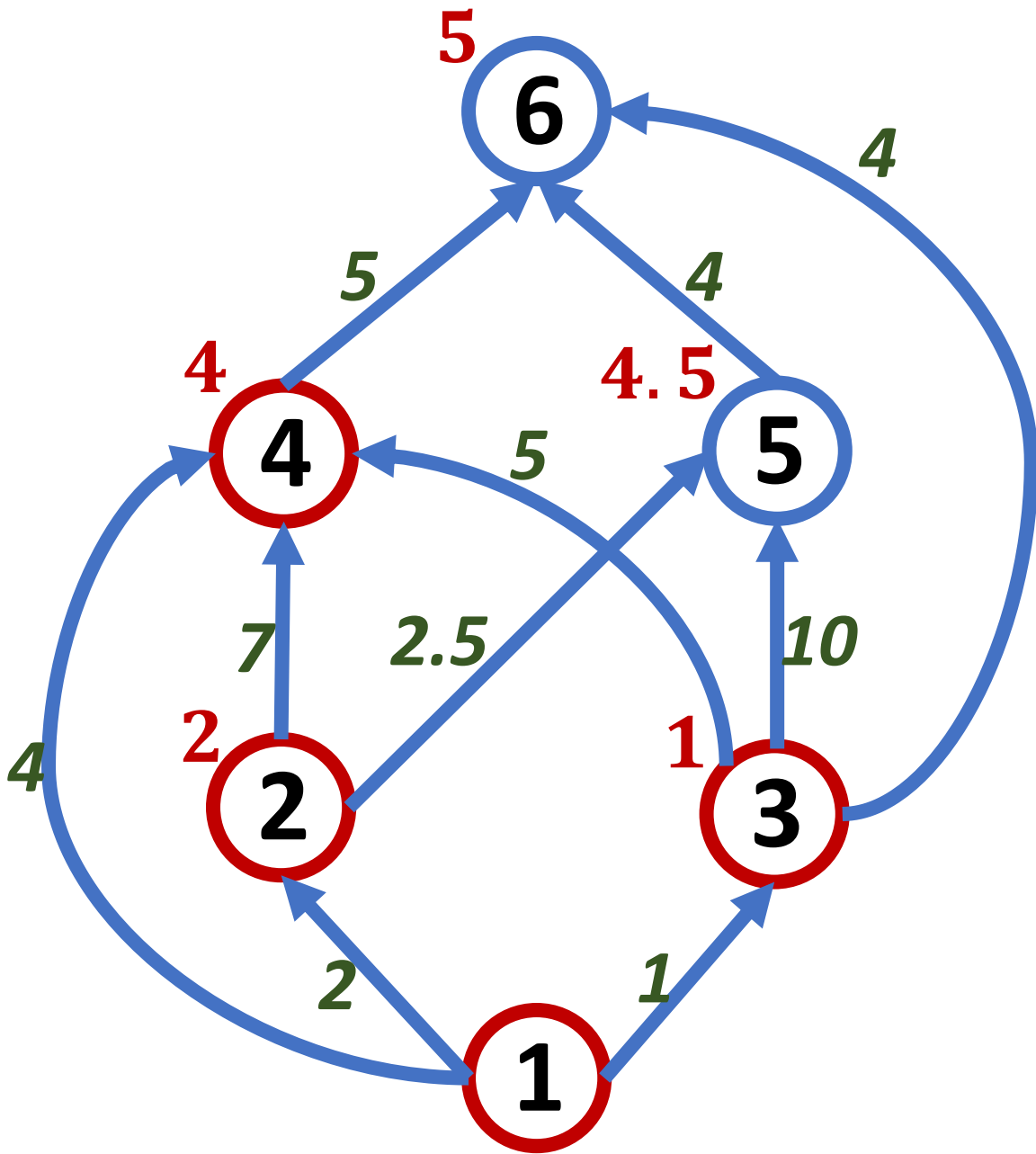
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
0	1	1	1	2	3



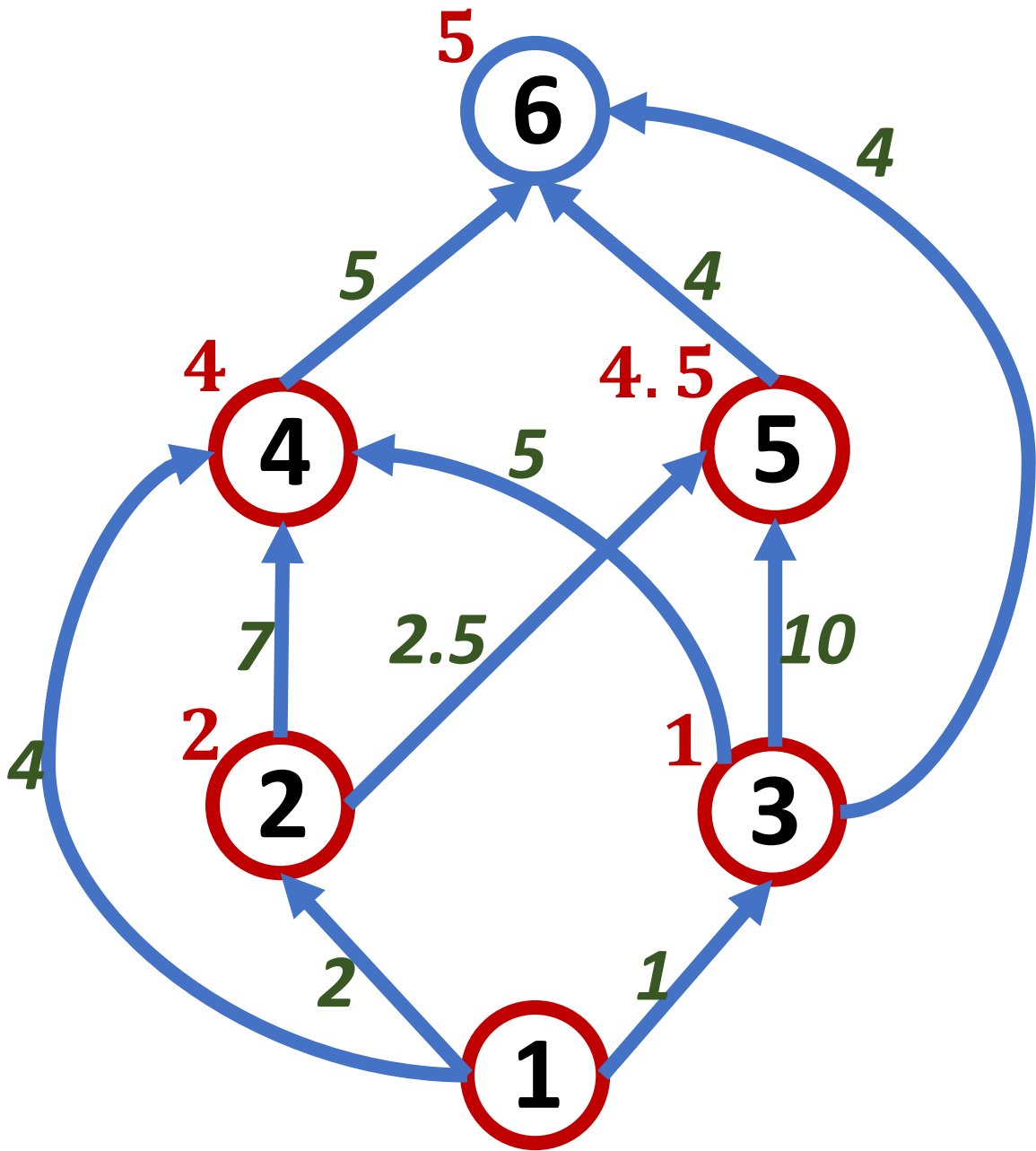
1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
0	1	1	1	2	3



1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
					4+5>5
0	1	1	1	2	3

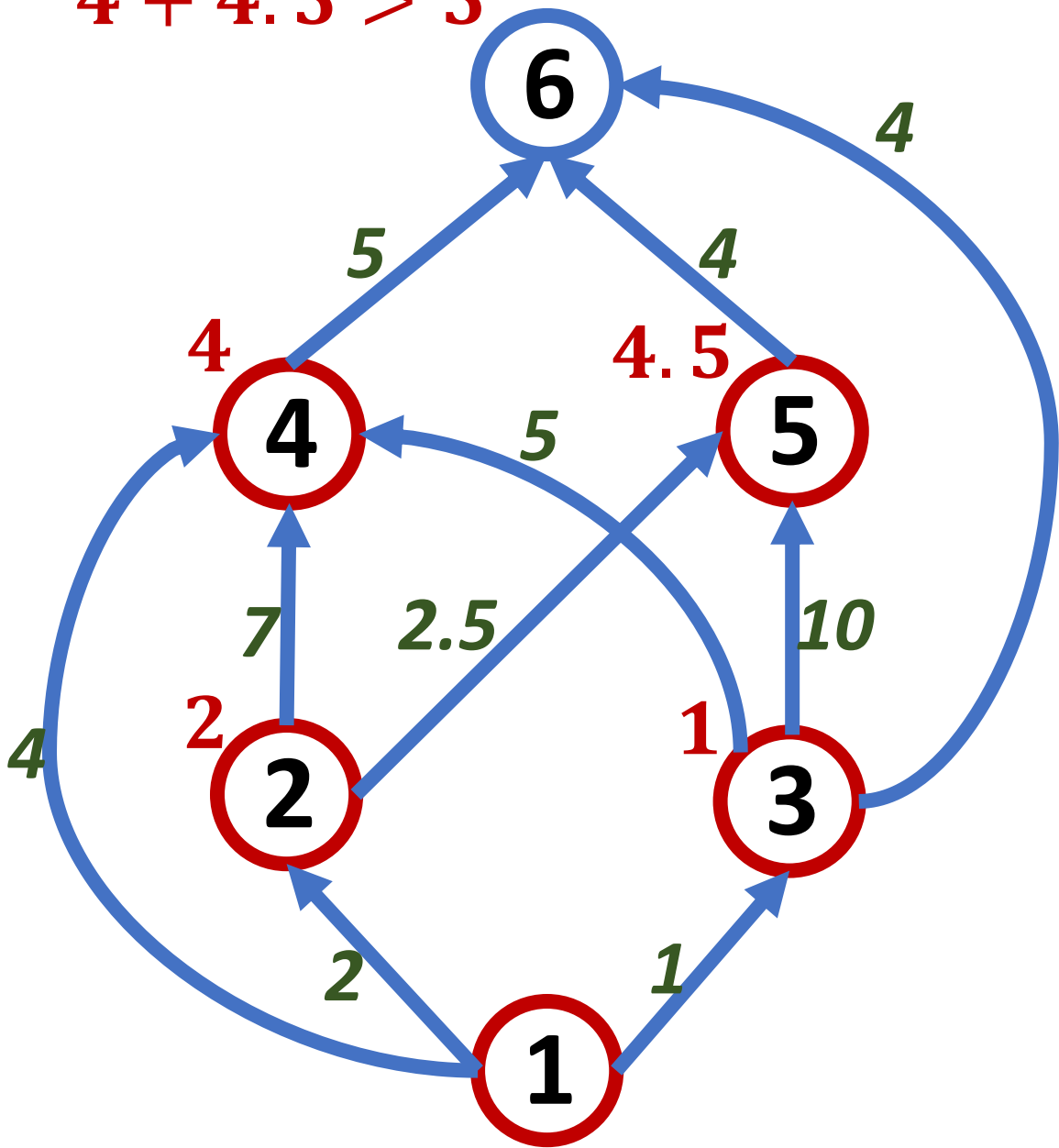


1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
				4.5	5
0	1	1	1	2	3

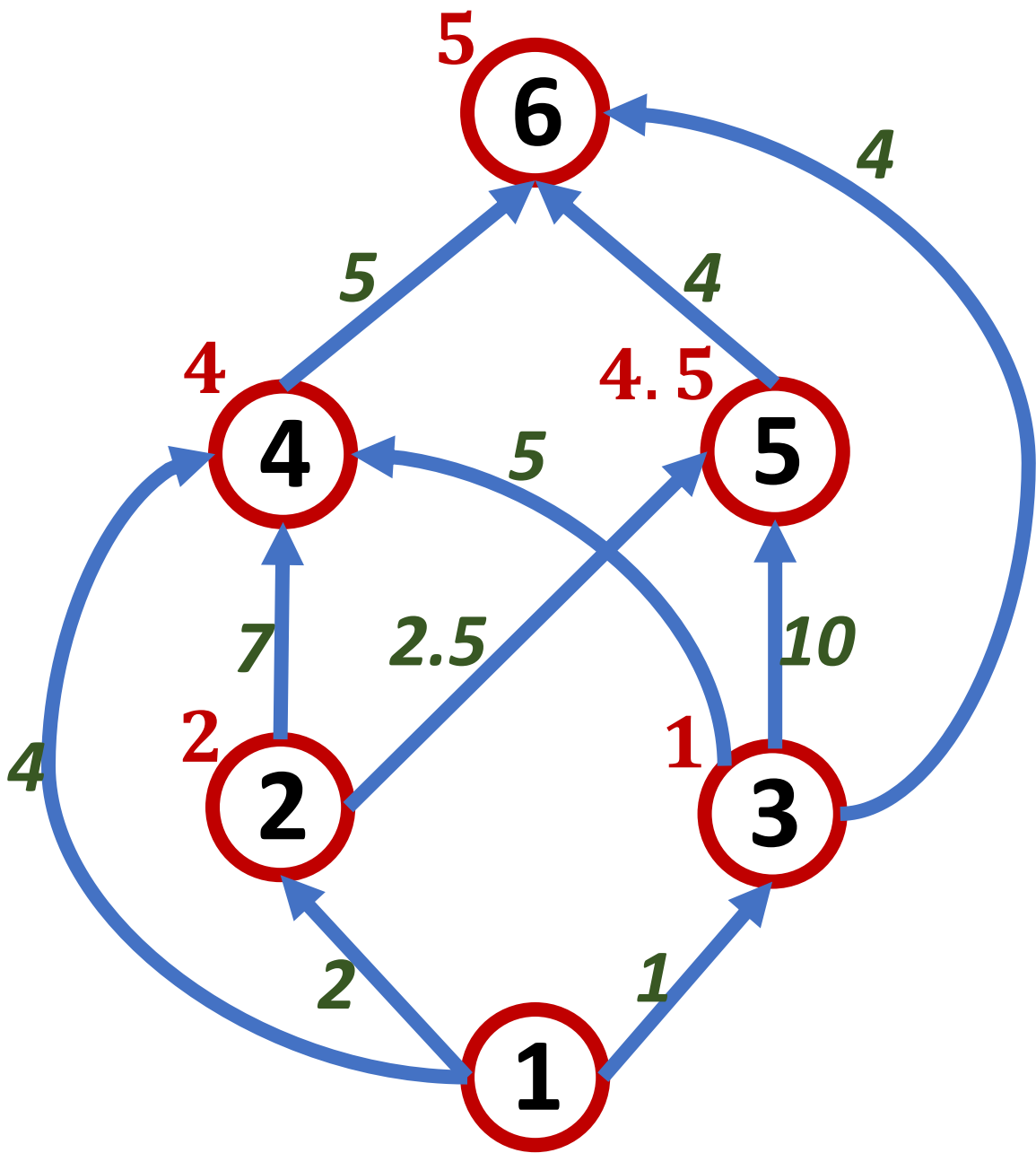


1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
				4.5	5
0	1	1	1	2	3

$$4 + 4.5 > 5$$

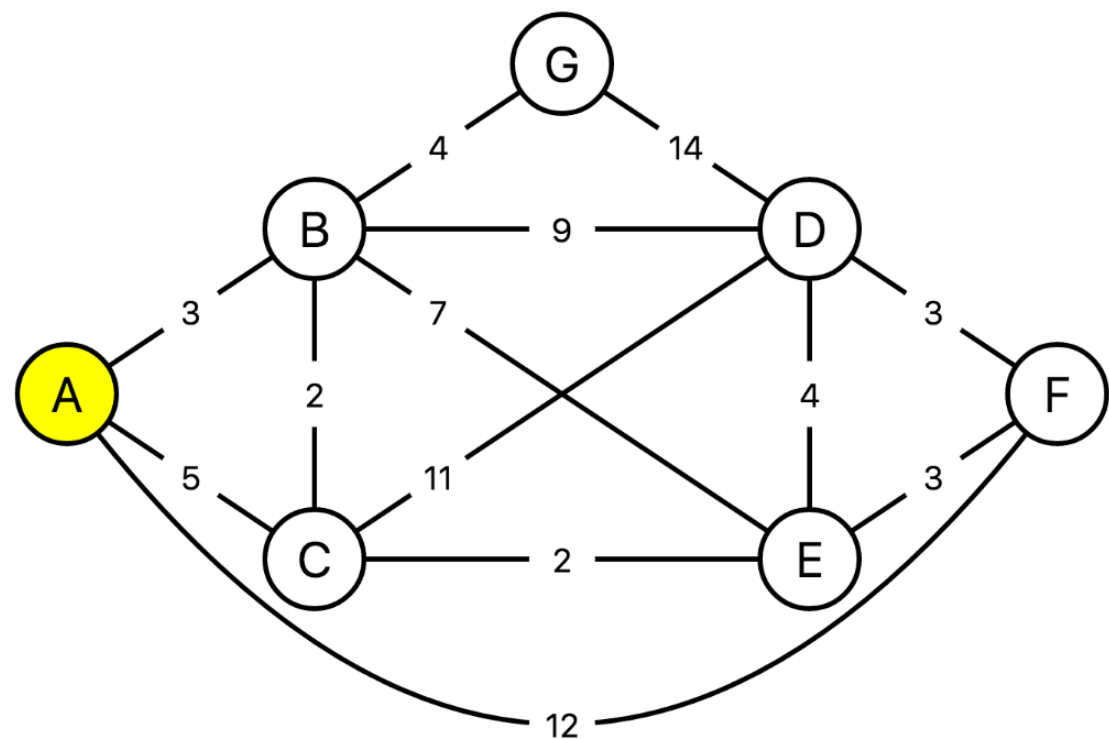


1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
				4.5	5
					4+4.5 >5
0	1	1	1	2	20 3



1	2	3	4	5	6
0	∞	∞	∞	∞	∞
	2	1	4	∞	∞
	2		4	11	5
			4	4.5	5
				4.5	5
					5
0	1	1	1	2	3

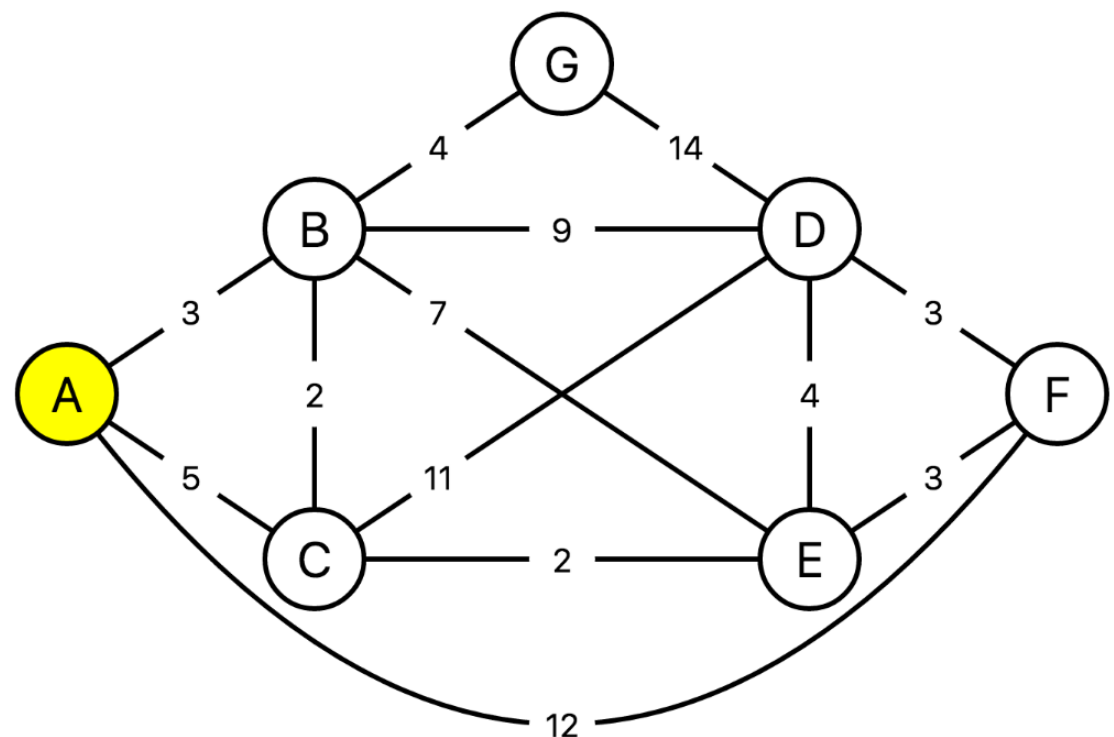
Найти кратчайший маршрут из
вершины A в F



(cs.manchester.ac.uk)

A	B	C	D	E	F	G

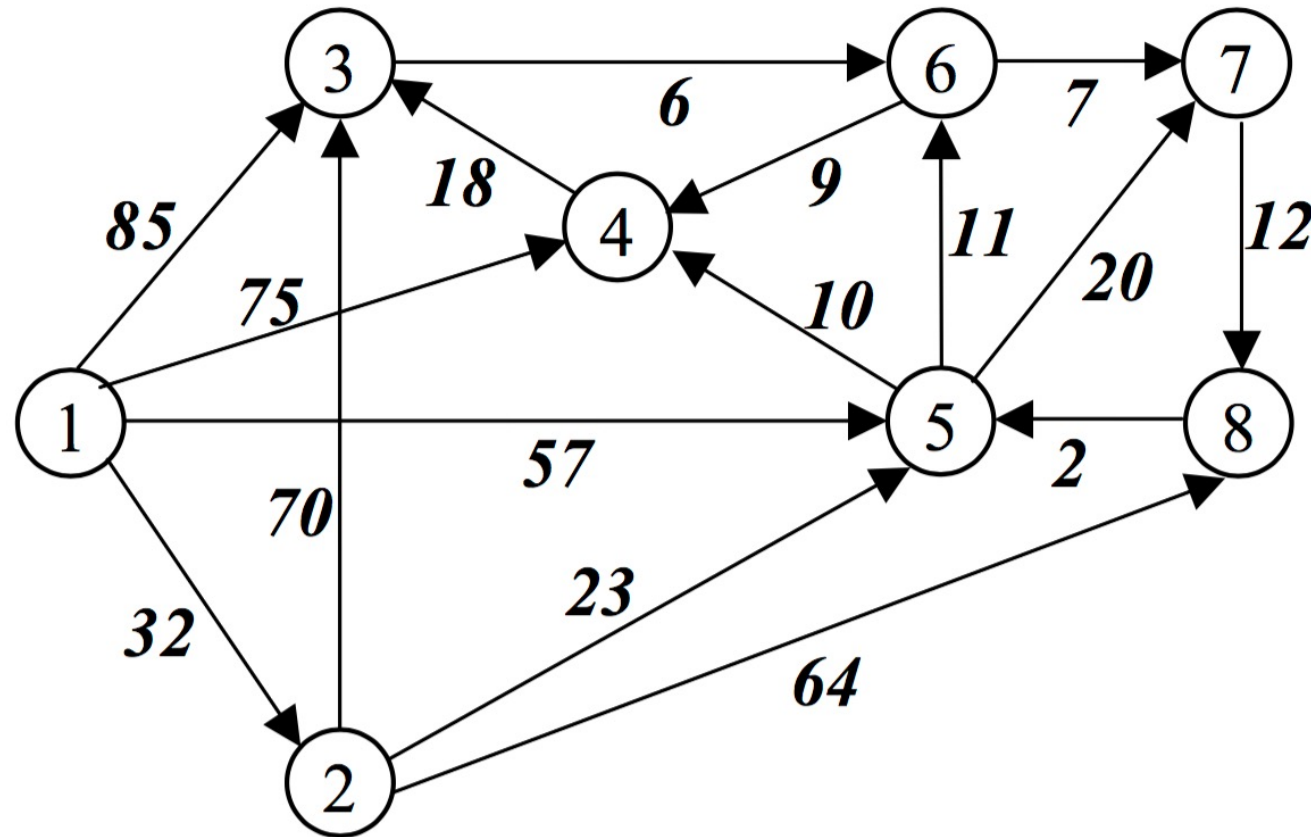
Найти кратчайший маршрут из
вершины A в F



(cs.manchester.ac.uk)

A	B	C	D	E	F	G
0	∞	∞	∞	∞	∞	∞
	3	5	∞	∞	12	∞
		5	12	10	12	7
			12	7	12	7
			11		10	7
			11		10	
-	A	A	E	C	E	B

Найти кратчайшие пути от первой вершины ко всем остальным, используя алгоритм Дейкстры.



Вычислительная сложность алгоритма Дейкстры

Напомним, что сложность алгоритма Дейкстры складывается из двух основных операций: время нахождения вершины с наименьшей величиной расстояния $d[v]$, и время совершения релаксации, т.е. время изменения величины $d[to]$.

При простейшей реализации эти операции потребуют соответственно $O(n)$ и $O(1)$ времени. Учитывая, что первая операция всего выполняется $O(n)$ раз, а вторая — $O(m)$, получаем асимптотику простейшей реализации алгоритма Дейкстры: $O(n^2 + m)$.

Простейшая реализация:
хранение в массиве,
сложность $O(n^2 + m)$

Восстановление пути:

```
vector<int> path;  
for (int v=t; v!=s; v=p[v])  
    path.push_back (v);  
path.push_back (s);  
reverse (path.begin(), path.end());
```

```
const int INF = 1000000000;  
  
int main() {  
    int n;  
    ... чтение n ...  
    vector < vector < pair<int,int> > > g (n);  
    ... чтение графа ...  
    int s = ...; // стартовая вершина  
  
    vector<int> d (n, INF), p (n);  
    d[s] = 0;  
    vector<char> u (n);  
    for (int i=0; i<n; ++i) {  
        int v = -1;  
        for (int j=0; j<n; ++j)  
            if (!u[j] && (v == -1 || d[j] < d[v]))  
                v = j;  
        if (d[v] == INF)  
            break;  
        u[v] = true;  
  
        for (size_t j=0; j<g[v].size(); ++j) {  
            int to = g[v][j].first,  
                len = g[v][j].second;  
            if (d[v] + len < d[to]) {  
                d[to] = d[v] + len;  
                p[to] = v;  
            }  
        }  
    }  
}
```

Вычислительная сложность алгоритма Дейкстры

При простейшей реализации эти операции потребуют соответственно $O(n)$ и $O(1)$ времени. Учитывая, что первая операция всего выполняется $O(n)$ раз, а вторая — $O(m)$, получаем асимптотику простейшей реализации алгоритма Дейкстры: $O(n^2 + m)$.

Понятно, что эта асимптотика является оптимальной для плотных графов, т.е. когда $m \approx n^2$. Чем более разрежен граф (т.е. чем меньше m по сравнению с максимальным количеством рёбер n^2), тем менее оптимальной становится эта оценка, и по вине первого слагаемого. Таким образом, надо улучшать время выполнения операций первого типа, не сильно ухудшая при этом время выполнения операций второго типа.

Для этого надо использовать различные вспомогательные структуры данных. Наиболее привлекательными являются **Фибоначчиевы кучи**, которые позволяют производить операцию первого вида за $O(\log n)$, а второго — за $O(1)$. Поэтому при использовании Фибоначчиевых куч время работы алгоритма Дейкстры составит $O(n \log n + m)$, что является практически теоретическим минимумом для алгоритма

Фибоначчиевы кучи

[Кратко] Фибоначчиева куча — набор из подвешенных деревьев, удовлетворяющих свойству: каждый предок не больше своих детей.

Недостатки:

- Большое потребление памяти на узел(минимум 21 байт)
- Большая константа времени работы, что делает ее малоприменимой для реальных задач
- Некоторые операции в худшем случае могут работать за $O(n)$ времени

Достоинства:

- Одно из лучших асимптотических времен работы для всех операций

Пример реализации данной структуры, состоящий из 290 строк...

<http://www.everfall.com/paste/id.php?696pwn56g065>

Вычислительная сложность алгоритма Дейкстры

В качестве компромисса можно использовать структуры данных, позволяющие выполнять **оба типа операций** (фактически, это извлечение минимума и обновление элемента) за $O(\log n)$. Тогда время работы алгоритма Дейкстры составит:

$$O(n \log n + m \log n) = O(m \log n)$$

В качестве такой структуры данных программистам на C++ удобно взять стандартный контейнер `set` или `priority_queue`. Первый основан на красно-чёрном дереве, второй — на бинарной куче. Поэтому `priority_queue` имеет меньшую константу, скрытую в асимптотике, однако у него есть и недостаток: он не поддерживает операцию удаления элемента, из-за чего приходится делать "обходной манёвр", который фактически приводит к замене в асимптотике $\log n$ на $\log m$ (с точки зрения асимптотики это на самом деле ничего не меняет, но скрытую константу увеличивает).

Реализация через set:
в контейнер помещаются пары (**расстояние, номер вершины**), которые будут автоматически отсортированы.

Массив `used` нам теперь больше не нужен.

Перед выполнением каждой успешной релаксации мы сначала удаляем старую пару, а затем, после выполнения релаксации, добавляем обратно новую пару (с новым расстоянием `d[to]`)

```
const int INF = 1000000000;
```

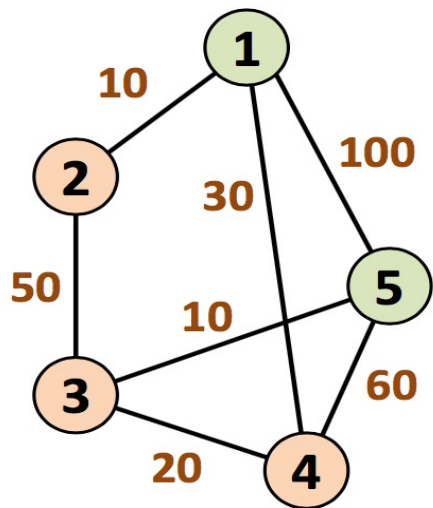
https://e-maxx.ru/algo/dijkstra_sparse

```
int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    set < pair<int,int> > q;
    q.insert (make_pair (d[s], s));
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase (q.begin());

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                q.erase (make_pair (d[to], to));
                d[to] = d[v] + len;
                p[to] = v;
                q.insert (make_pair (d[to], to));
            }
        }
    }
}
```

Найти кратчайший путь из вершины 1 в вершину 5



1	2	3	4	5	d[s]	s

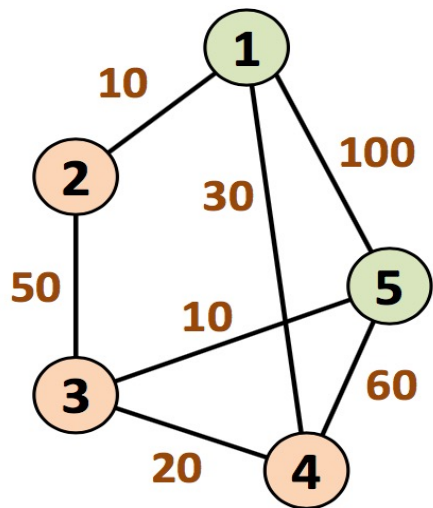
```
const int INF = 1000000000;

int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    set < pair<int,int> > q;
    q.insert (make_pair (d[s], s));
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase (q.begin());

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                q.erase (make_pair (d[to], to));
                d[to] = d[v] + len;
                p[to] = v;
                q.insert (make_pair (d[to], to));
            }
        }
    }
}
```

Найти кратчайший путь из вершины 1 в вершину 5



1	2	3	4	5	d[s]	s
0	∞	∞	∞	∞		
	10	∞	30	100		
		60	30	100		
		50		90		
				60		
0	1	4	1	3		

```
const int INF = 1000000000;
```

```
int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    set < pair<int,int> > q;
    q.insert (make_pair (d[s], s));
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase (q.begin());

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                q.erase (make_pair (d[to], to));
                d[to] = d[v] + len;
                p[to] = v;
                q.insert (make_pair (d[to], to));
            }
        }
    }
}
```


Реализация через priority_queue:

в контейнер помещаются пары (-расстояние, номер вершины).

Почему? 😊

Удалять произвольные элементы невозможно, поэтому приходится совершать "обходной манёвр": при релаксации просто не будем удалять старые пары из очереди. В результате в очереди могут находиться одновременно несколько пар для одной и той же вершины (но с разными расстояниями). Среди этих пар нас интересует только одна, для которой элемент first = d[v]. На практике версия с priority_queue оказывается несколько быстрее версии с set

```
const int INF = 1000000000;
```

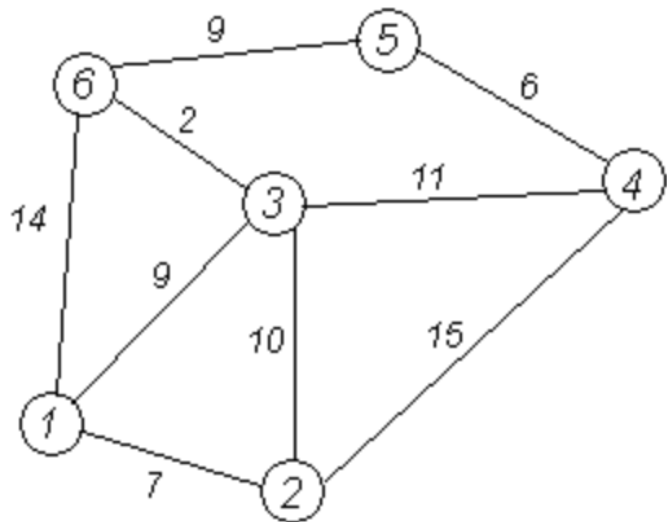
https://e-maxx.ru/algo/dijkstra_sparse

```
int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    priority_queue < pair<int,int> > q;
    q.push (make_pair (0, s));
    while (!q.empty()) {
        int v = q.top().second, cur_d = -q.top().first;
        q.pop();
        if (cur_d > d[v]) continue;

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
                q.push (make_pair (-d[to], to));
            }
        }
    }
}
```

Найти кратчайший путь из вершины 1 в вершину 6



1	2	3	4	5	6	-d[s]	s

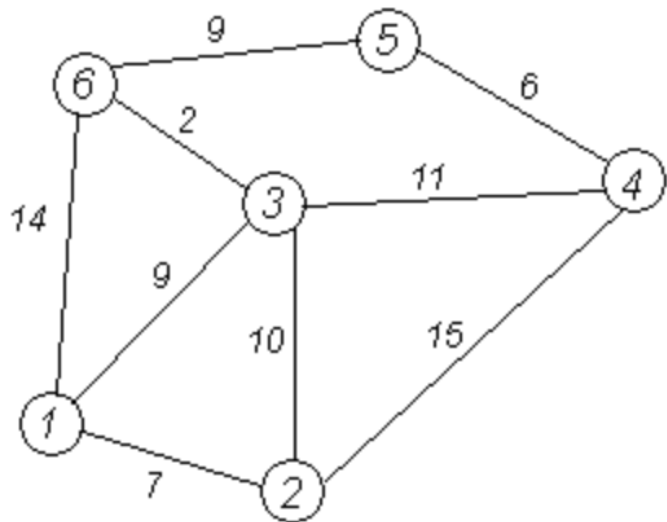
```
const int INF = 1000000000;

int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    priority_queue < pair<int,int> > q;
    q.push (make_pair (0, s));
    while (!q.empty()) {
        int v = q.top().second, cur_d = -q.top().first;
        q.pop();
        if (cur_d > d[v]) continue;

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
                q.push (make_pair (-d[to], to));
            }
        }
    }
}
```

Найти кратчайший путь из вершины 1 в вершину 6



1	2	3	4	5	6	-d[s]	s
0	∞	∞	∞	∞	∞		
	7	9	∞	∞	14		
		9	22	∞	14		
			20	∞	11		
			20	20			
0	1	1	3	6	3		

```
const int INF = 1000000000;
```

```
int main() {
    int n;
    ... чтение n ...
    vector < vector < pair<int,int> > > g (n);
    ... чтение графа ...
    int s = ...; // стартовая вершина

    vector<int> d (n, INF), p (n);
    d[s] = 0;
    priority_queue < pair<int,int> > q;
    q.push (make_pair (0, s));
    while (!q.empty()) {
        int v = q.top().second, cur_d = -q.top().first;
        q.pop();
        if (cur_d > d[v]) continue;

        for (size_t j=0; j<g[v].size(); ++j) {
            int to = g[v][j].first,
                len = g[v][j].second;
            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
                q.push (make_pair (-d[to], to));
            }
        }
    }
}
```

Вычислительная сложность алгоритма Дейкстры

Вариант реализации алгоритма Дейкстры	Плотный граф $m = O(n^2)$	Разреженный граф $m = O(n)$
$d[i]$ – это массив (поиск за время $O(n)$)	$T = O(n^2 + m) = O(n^2)$	$T = O(n^2 + m) = O(n^2)$
$d[i]$ хранятся в бинарной куче	$T = O(n \log n + m \log n) = O(n^2 \log n)$	$T = O(n \log n + m \log n) = O(n \log n)$
$d[i]$ хранятся в Фибоначчиевой куче	$T = O(m + n \log n) = O(n^2)$	$T = O(m + n \log n) = O(n \log n)$

Алгоритм Флойда-Уоршелла

Алгоритм Флойда – Уоршелла – динамический алгоритм вычисления значений кратчайших путей для каждой из вершин графа. Метод работает на взвешенных графах, с положительными и отрицательными весами ребер, но без отрицательных циклов, являясь, таким образом, более общим в сравнении с алгоритмом Дейкстры, т. к. последний не работает с отрицательными весами ребер.

Алгоритм Флойда-Уоршелла

Пусть вершины графа $G = (V, E)$, $|V| = n$ пронумерованы от 1 до n и введено обозначение d_{ij}^k для длины кратчайшего пути от i до j , который кроме самих вершин i, j проходит только через вершины $1 \dots k$. Очевидно, что d_{ij}^0 — длина (вес) ребра (i, j) , если таковое существует (в противном случае его длина может быть обозначена как ∞).

Существует два варианта значения d_{ij}^k , $k \in (1, \dots, n)$:

1. Кратчайший путь между i, j не проходит через вершину k , тогда $d_{ij}^k = d_{ij}^{k-1}$
2. Существует более короткий путь между i, j , проходящий через k , тогда он сначала идёт от i до k , а потом от k до j . В этом случае, очевидно, $d_{ij}^k = d_{ik}^{k-1} + d_{kj}^{k-1}$

Таким образом, для нахождения значения функции достаточно выбрать минимум из двух обозначенных значений.

Тогда **рекуррентная** формула для d_{ij}^k имеет вид:

d_{ij}^0 — длина ребра (i, j) ;

$$d_{ij}^k = \min(d_{ij}^{k-1}, d_{ik}^{k-1} + d_{kj}^{k-1}).$$

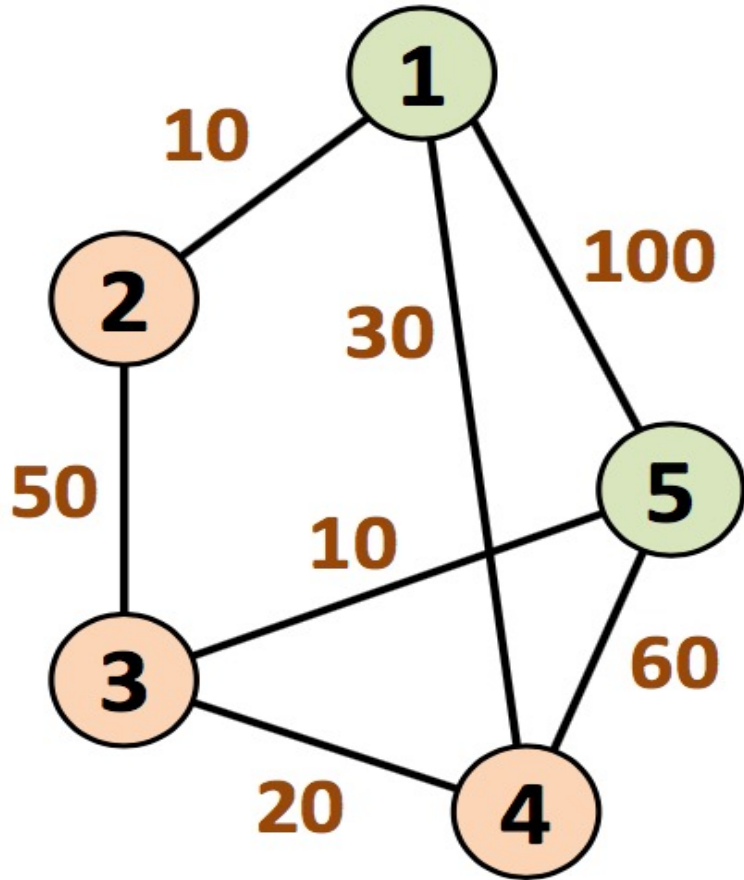
На вход программе подаётся граф, заданный в виде матрицы смежности — двумерного массива $d[] []$ размера $n \times n$, в котором каждый элемент задаёт длину ребра между соответствующими вершинами.

Требуется, чтобы выполнялось $d[i][i] = 0$ для любых i .

```
for (int k=0; k<n; ++k)
    for (int i=0; i<n; ++i)
        for (int j=0; j<n; ++j)
            d[i][j] = min (d[i][j], d[i][k] + d[k][j]);
```

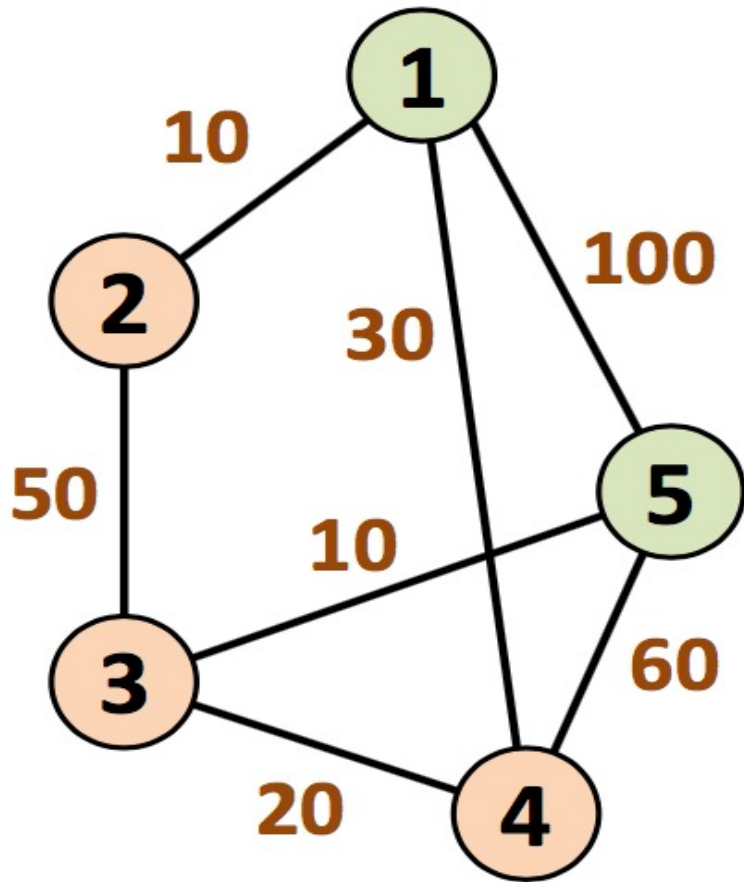
Предполагается, что если между двумя какими-то вершинами **нет ребра**, то в матрице смежности было записано какое-то большое число (достаточно большое, чтобы оно было больше длины любого пути в этом графе); тогда это ребро всегда будет невыгодно брать, и алгоритм сработает правильно.

Необходимо найти кратчайшие пути между каждой парой вершин в графе, представленном на рисунке



	1	2	3	4	5
1					
2					
3					
4					
5					

Необходимо найти кратчайшие пути между каждой парой вершин в графе, представленном на рисунке



	1	2	3	4	5
1	0	10	∞	30	100
2	10	0	50	∞	∞
3	∞	50	0	20	10
4	30	∞	20	0	60
5	100	∞	10	60	0

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

Например, $k = 1$

$$d[1][1] = \min(d[1][1], d[1][1] + d[1][1])$$

...

$$d[1][5] = \min(d[1][5], d[1][1] + d[5][1])$$

$$d[2][2] = \min(d[2][2], d[2][1] + d[1][2])$$

$$d[2][3] = \min(d[2][3], d[2][1] + d[1][3])$$

$$d[2][4] = \min(d[2][4], d[2][1] + d[1][4])$$

$$d[2][5] = \min(d[2][5], d[2][1] + d[1][5])$$

$$d[3][3] = \min(d[3][3], d[3][1] + d[1][3])$$

$$d[3][4] = \min(d[3][4], d[3][1] + d[1][4])$$

$$d[3][5] = \min(d[3][5], d[3][1] + d[1][5])$$

$$d[4][4] = \min(d[4][4], d[4][1] + d[1][4])$$

$$d[4][5] = \min(d[4][5], d[4][1] + d[1][5])$$

$$d[5][5] = \min(d[5][5], d[5][1] + d[1][5])$$

	1	2	3	4	5
1	0	10	∞	30	100
2	10	0	50	∞	∞
3	∞	50	0	20	10
4	30	∞	20	0	60
5	100	∞	10	60	0

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

Например, $k = 1$

$$d[1][1] = \min(d[1][1], d[1][1] + d[1][1])$$

...

$$d[1][5] = \min(d[1][5], d[1][1] + d[5][1])$$

$$d[2][2] = \min(d[2][2], d[2][1] + d[1][2])$$

$$d[2][3] = \min(d[2][3], d[2][1] + d[1][3])$$

$$d[2][4] = \min(d[2][4], d[2][1] + d[1][4])$$

$$d[2][5] = \min(d[2][5], d[2][1] + d[1][5])$$

$$d[3][3] = \min(d[3][3], d[3][1] + d[1][3])$$

$$d[3][4] = \min(d[3][4], d[3][1] + d[1][4])$$

$$d[3][5] = \min(d[3][5], d[3][1] + d[1][5])$$

$$d[4][4] = \min(d[4][4], d[4][1] + d[1][4])$$

$$d[4][5] = \min(d[4][5], d[4][1] + d[1][5])$$

$$d[5][5] = \min(d[5][5], d[5][1] + d[1][5])$$

	1	2	3	4	5
1	0	10	∞	30	100
2	10	0	50	40	110
3	∞	50	0	20	10
4	30	40	20	0	60
5	100	110	10	60	0

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

А теперь, $k = 2$

$$d[1][1] = \min(d[1][1], d[1][2] + d[2][1])$$

$$d[1][2] = \min(d[1][2], d[1][2] + d[2][2])$$

$$d[1][3] = \min(d[1][3], d[1][2] + d[2][3])$$

$$d[1][4] = \min(d[1][4], d[1][2] + d[2][4])$$

$$d[1][5] = \min(d[1][5], d[1][2] + d[2][5])$$

$$d[2][2] = \min(d[2][2], d[2][2] + d[2][2])$$

...

$$d[2][5] = \min(d[2][5], d[2][2] + d[2][5])$$

$$d[3][3] = \min(d[3][3], d[3][2] + d[2][3])$$

$$d[3][4] = \min(d[3][4], d[3][2] + d[2][4])$$

$$d[3][5] = \min(d[3][5], d[3][2] + d[2][5])$$

$$d[4][4] = \min(d[4][4], d[4][2] + d[2][4])$$

$$d[4][5] = \min(d[4][5], d[4][2] + d[2][5])$$

$$d[5][5] = \min(d[5][5], d[5][2] + d[2][5])$$

	1	2	3	4	5
1	0	10	∞	30	100
2	10	0	50	40	110
3	∞	50	0	20	10
4	30	40	20	0	60
5	100	110	10	60	0

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

А теперь, $k = 2$

$$d[1][1] = \min(d[1][1], d[1][2] + d[2][1])$$

$$d[1][2] = \min(d[1][2], d[1][2] + d[2][2])$$

$$\mathbf{d[1][3] = \min(d[1][3], d[1][2] + d[2][3])}$$

$$d[1][4] = \min(d[1][4], d[1][2] + d[2][4])$$

$$d[1][5] = \min(d[1][5], d[1][2] + d[2][5])$$

$$d[2][2] = \min(d[2][2], d[2][2] + d[2][2])$$

...

$$d[2][5] = \min(d[2][5], d[2][2] + d[2][5])$$

$$d[3][3] = \min(d[3][3], d[3][2] + d[2][3])$$

$$d[3][4] = \min(d[3][4], d[3][2] + d[2][4])$$

$$d[3][5] = \min(d[3][5], d[3][2] + d[2][5])$$

$$d[4][4] = \min(d[4][4], d[4][2] + d[2][4])$$

$$d[4][5] = \min(d[4][5], d[4][2] + d[2][5])$$

$$d[5][5] = \min(d[5][5], d[5][2] + d[2][5])$$

	1	2	3	4	5
1	0	10	60	30	100
2	10	0	50	40	110
3	60	50	0	20	10
4	30	40	20	0	60
5	100	110	10	60	0

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

А если $k = 3$?

$d[1][2] = \min(d[1][2], d[1][3] + d[3][2])$
 $d[1][3] = \min(d[1][3], d[1][3] + d[3][3])$
 $d[1][4] = \min(d[1][4], d[1][3] + d[3][4])$
 $d[1][5] = \min(d[1][5], d[1][3] + d[3][5])$

$d[2][3] = \min(d[2][3], d[2][3] + d[3][3])$
 $d[2][4] = \min(d[2][4], d[2][3] + d[3][4])$
 $d[2][5] = \min(d[2][5], d[2][3] + d[3][5])$

$d[3][4] = \min(d[3][4], d[3][3] + d[3][4])$
 $d[3][5] = \min(d[3][5], d[3][3] + d[3][5])$

$d[4][5] = \min(d[4][5], d[4][3] + d[3][5])$

	1	2	3	4	5
1	0	10	60	30	100
2	10	0	50	40	110
3	60	50	0	20	10
4	30	40	20	0	60
5	100	110	10	60	0

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

А если $k = 3$?

$d[1][2] = \min(d[1][2], d[1][3] + d[3][2])$
 $d[1][3] = \min(d[1][3], d[1][3] + d[3][3])$
 $d[1][4] = \min(d[1][4], d[1][3] + d[3][4])$
 $d[1][5] = \min(d[1][5], d[1][3] + d[3][5])$

$d[2][3] = \min(d[2][3], d[2][3] + d[3][3])$
 $d[2][4] = \min(d[2][4], d[2][3] + d[3][4])$
 $d[2][5] = \min(d[2][5], d[2][3] + d[3][5])$

$d[3][4] = \min(d[3][4], d[3][3] + d[3][4])$
 $d[3][5] = \min(d[3][5], d[3][3] + d[3][5])$

$d[4][5] = \min(d[4][5], d[4][3] + d[3][5])$

	1	2	3	4	5
1	0	10	60	30	70
2	10	0	50	40	60
3	60	50	0	20	10
4	30	40	20	0	30
5	70	60	10	30	0

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

Что насчет $k = 4$?

$d[1][2] = \min(d[1][2], d[1][4] + d[4][2])$
 $d[1][3] = \min(d[1][3], d[1][4] + d[4][3])$
 $d[1][4] = \min(d[1][4], d[1][4] + d[4][4])$
 $d[1][5] = \min(d[1][5], d[1][4] + d[4][5])$

$d[2][3] = \min(d[2][3], d[2][4] + d[4][3])$
 $d[2][4] = \min(d[2][4], d[2][4] + d[4][4])$
 $d[2][5] = \min(d[2][5], d[2][4] + d[4][5])$

$d[3][4] = \min(d[3][4], d[3][4] + d[4][4])$
 $d[3][5] = \min(d[3][5], d[3][4] + d[4][5])$

$d[4][5] = \min(d[4][5], d[4][4] + d[4][5])$

	1	2	3	4	5
1	0	10	60	30	70
2	10	0	50	40	60
3	60	50	0	20	10
4	30	40	20	0	30
5	70	60	10	30	0

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

Что насчет $k = 4$?

$$d[1][2] = \min(d[1][2], d[1][4] + d[4][2])$$

$$d[1][3] = \min(d[1][3], d[1][4] + d[4][3])$$

$$d[1][4] = \min(d[1][4], d[1][4] + d[4][4])$$

$$d[1][5] = \min(d[1][5], d[1][4] + d[4][5])$$

$$d[2][3] = \min(d[2][3], d[2][4] + d[4][3])$$

$$d[2][4] = \min(d[2][4], d[2][4] + d[4][4])$$

$$d[2][5] = \min(d[2][5], d[2][4] + d[4][5])$$

$$d[3][4] = \min(d[3][4], d[3][4] + d[4][4])$$

$$d[3][5] = \min(d[3][5], d[3][4] + d[4][5])$$

$$d[4][5] = \min(d[4][5], d[4][4] + d[4][5])$$

	1	2	3	4	5
1	0	10	50	30	60
2	10	0	50	40	60
3	50	50	0	20	10
4	30	40	20	0	30
5	60	60	10	30	0

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

И напоследок, $k = 5$?

$d[1][2] = \min(d[1][2], d[1][5] + d[5][2])$
 $d[1][3] = \min(d[1][3], d[1][5] + d[5][3])$
 $d[1][4] = \min(d[1][4], d[1][5] + d[5][4])$
 $d[1][5] = \min(d[1][5], d[1][5] + d[5][5])$

$d[2][3] = \min(d[2][3], d[2][5] + d[5][3])$
 $d[2][4] = \min(d[2][4], d[2][5] + d[5][4])$
 $d[2][5] = \min(d[2][5], d[2][5] + d[5][5])$

$d[3][4] = \min(d[3][4], d[3][5] + d[5][4])$
 $d[3][5] = \min(d[3][5], d[3][5] + d[5][5])$

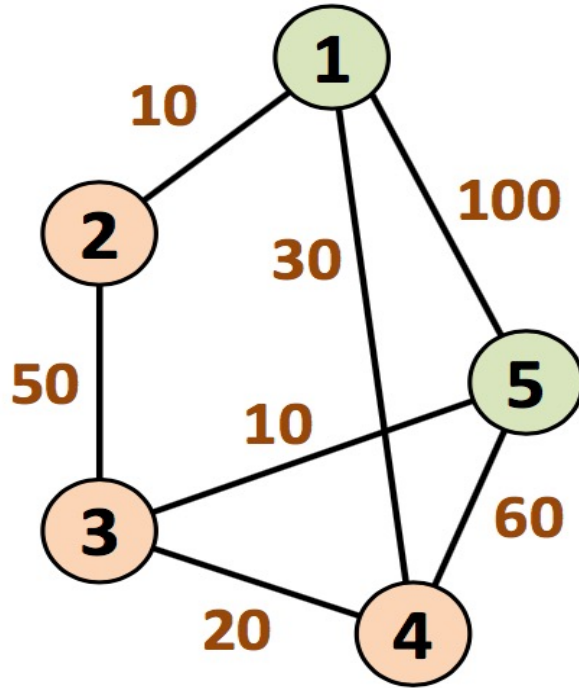
$d[4][5] = \min(d[4][5], d[4][5] + d[5][5])$

	1	2	3	4	5
1	0	10	50	30	60
2	10	0	50	40	60
3	50	50	0	20	10
4	30	40	20	0	30
5	60	60	10	30	0

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

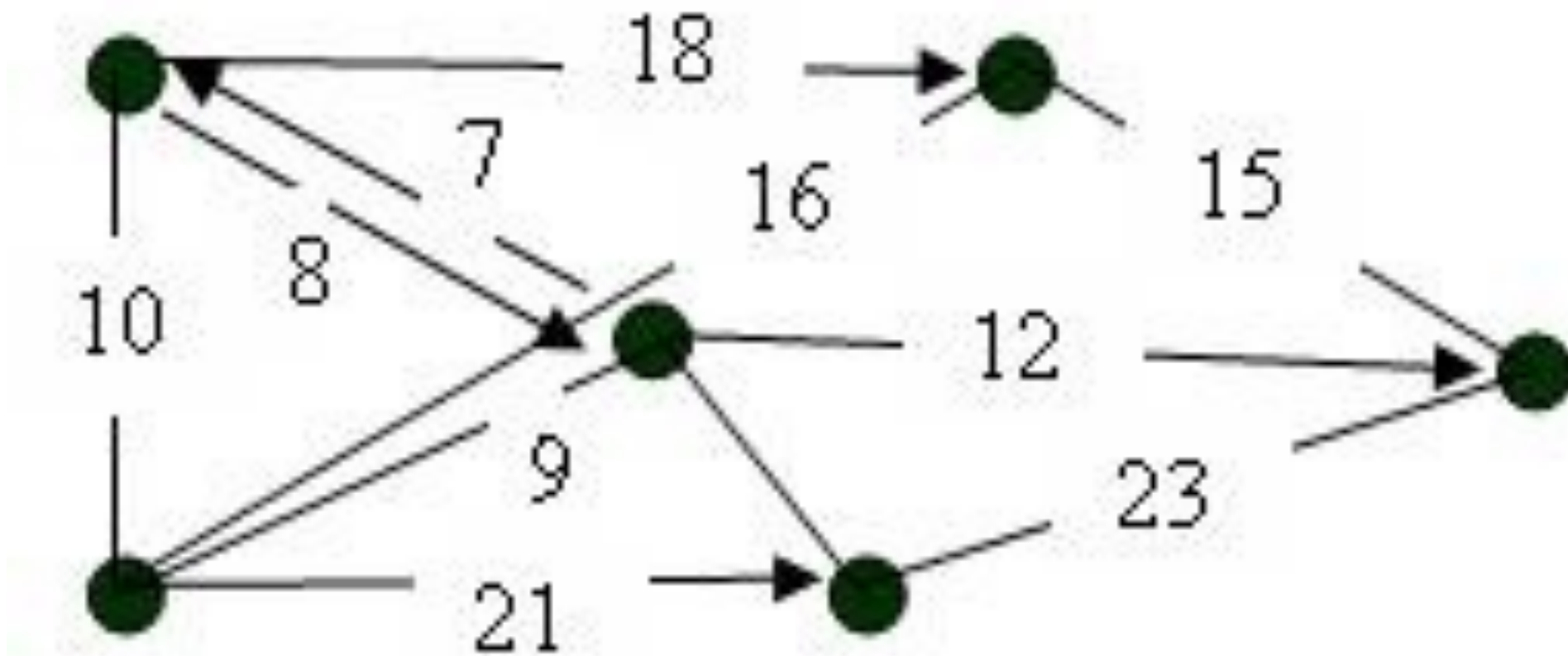
И напоследок, $k = 5$?

Менять нечего!

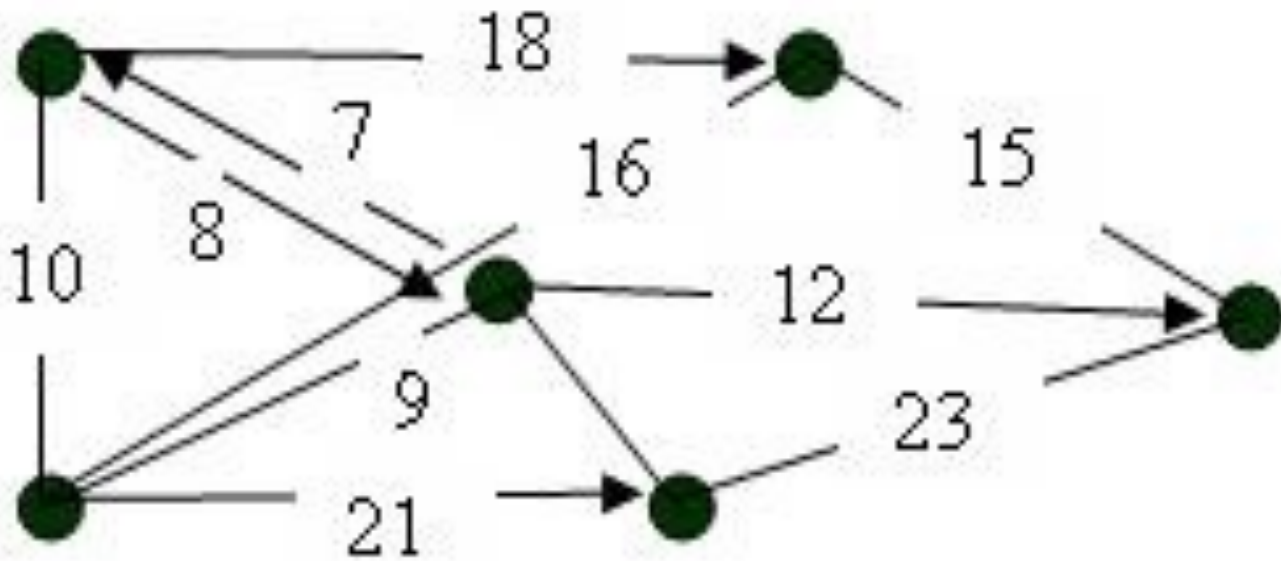


	1	2	3	4	5
1	0	10	50	30	60
2	10	0	50	40	60
3	50	50	0	20	10
4	30	40	20	0	30
5	60	60	10	30	0

Необходимо найти кратчайшие пути между каждой парой вершин в графе, представленном на рисунке



Ответ



$$D^6 =$$

0	10	18	8	31	20
10	0	16	9	21	21
26	16	0	25	37	15
7	9	25	0	30	12
64	54	38	63	0	23
41	31	15	40	23	0

Правда, если не принять специальных мер, то при наличии в графе рёбер **отрицательного веса**, в результирующей матрице могут появиться числа вида $\infty - 1$, $\infty - 2$, и т.д., которые, конечно, по-прежнему означают, что между соответствующими вершинами вообще нет пути. Поэтому при наличии в графе отрицательных рёбер алгоритм Флойда лучше написать так, чтобы он не выполнял переходы из тех состояний, в которых уже стоит "нет пути":

```
for (int k=0; k<n; ++k)
    for (int i=0; i<n; ++i)
        for (int j=0; j<n; ++j)
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min (d[i][j], d[i][k] + d[k][j]);
```

Случай вещественных весов

Если веса рёбер графа не целочисленные, а вещественные, то следует учитывать погрешности, неизбежно возникающие при работе с типами с плавающей точкой.

Применительно к алгоритму Флойда неприятным спецэффектом этих погрешностей становится то, что найденные алгоритмом расстояния могут уйти сильно в минус из-за **накопившихся ошибок**. В самом деле, если на первой фазе имела место ошибка Δ , то на второй итерации эта ошибка уже может превратиться в 2Δ , на третьей — в 4Δ , и так далее.

Чтобы этого не происходило, сравнения в алгоритме Флойда следует делать с учётом погрешности:

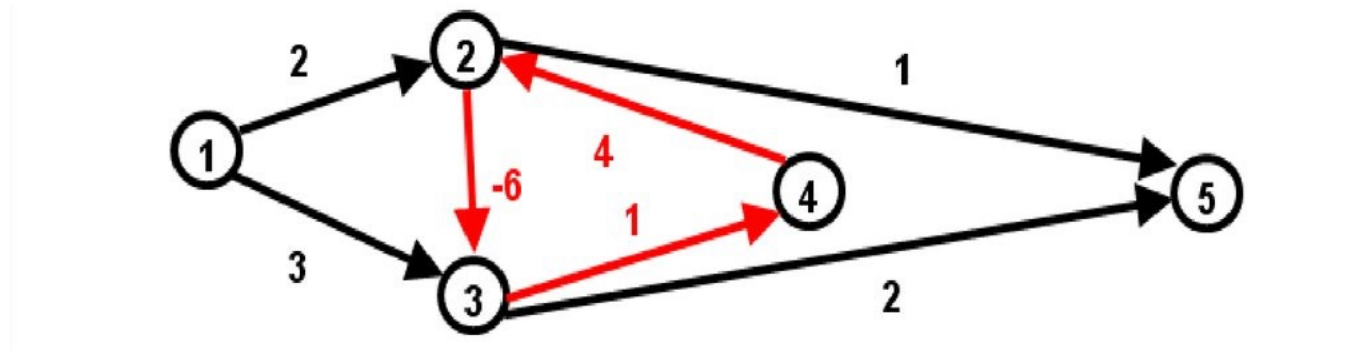
```
if (d[i][k] + d[k][j] < d[i][j] - EPS)
    d[i][j] = d[i][k] + d[k][j];
```

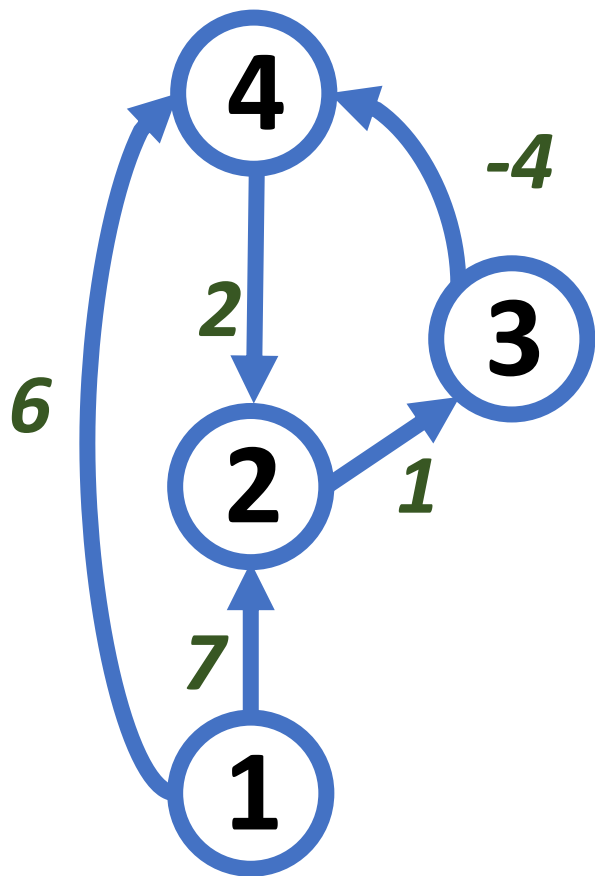
Случай отрицательных циклов

Если в графе есть циклы отрицательного веса, то формально алгоритм Флойда-Уоршелла неприменим к такому графу.

На самом же деле, для тех пар вершин i и j , между которыми нельзя зайти в цикл отрицательного веса, алгоритм отработает корректно.

Для тех же пар вершин, ответа для которых не существует (по причине наличия отрицательного цикла на пути между ними), алгоритм Флойда найдёт в качестве ответа какое-то число (возможно, сильно отрицательное, но не обязательно). Тем не менее, можно улучшить алгоритм Флойда, чтобы он аккуратно обрабатывал такие пары вершин и выводил для них, например, $-\infty$.





	1	2	3	4
1	0	7	∞	6
2	∞	0	1	∞
3	∞	∞	0	-4
4	∞	2	∞	0

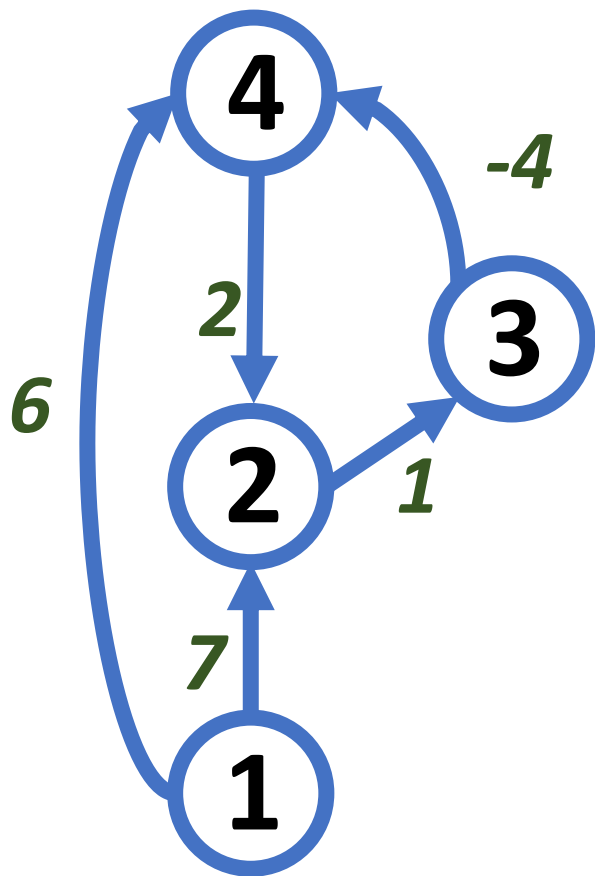
$k = 1$

$d[1][1] = \min(d[1][1], d[1][1] + d[1][1])$
 $d[1][2] = \min(d[1][2], d[1][1] + d[1][2])$
 $d[1][3] = \min(d[1][3], d[1][1] + d[1][3])$
 $d[1][4] = \min(d[1][4], d[1][1] + d[1][4])$

$d[2][1] = \min(d[2][1], d[2][1] + d[1][1])$
 $d[2][2] = \min(d[2][2], d[2][1] + d[1][2])$
 $d[2][3] = \min(d[2][3], d[2][1] + d[1][3])$
 $d[2][4] = \min(d[2][4], d[2][1] + d[1][4])$

$d[3][1] = \min(d[3][1], d[3][1] + d[1][1])$
 $d[3][2] = \min(d[3][2], d[3][1] + d[1][2])$
 $d[3][3] = \min(d[3][3], d[3][1] + d[1][3])$
 $d[3][4] = \min(d[3][4], d[3][1] + d[1][4])$

$d[4][1] = \min(d[4][1], d[4][1] + d[1][1])$
 $d[4][2] = \min(d[4][2], d[4][1] + d[1][2])$
 $d[4][3] = \min(d[4][3], d[4][1] + d[1][3])$
 $d[4][4] = \min(d[4][4], d[4][1] + d[1][4])$



	1	2	3	4
1	0	7	8	6
2	∞	0	1	∞
3	∞	∞	0	-4
4	∞	2	3	0

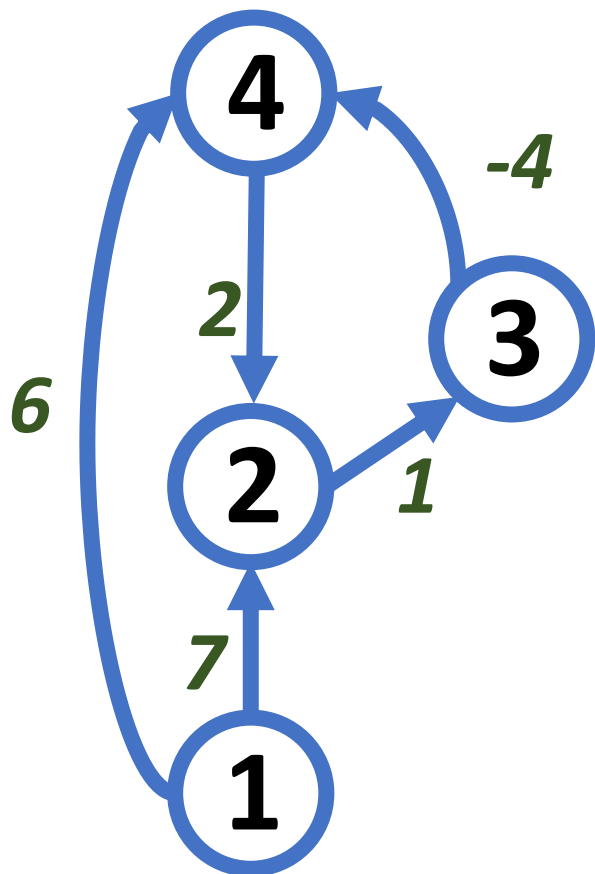
$k = 2$

$d[1][1] = \min(d[1][1], d[1][2] + d[2][1])$
 $d[1][2] = \min(d[1][2], d[1][2] + d[2][2])$
 $d[1][3] = \min(d[1][3], d[1][2] + d[2][3])$
 $d[1][4] = \min(d[1][4], d[1][2] + d[2][4])$

$d[2][1] = \min(d[2][1], d[2][2] + d[2][1])$
 $d[2][2] = \min(d[2][2], d[2][2] + d[2][2])$
 $d[2][3] = \min(d[2][3], d[2][2] + d[2][3])$
 $d[2][4] = \min(d[2][4], d[2][2] + d[2][4])$

$d[3][1] = \min(d[3][1], d[3][2] + d[2][1])$
 $d[3][2] = \min(d[3][2], d[3][2] + d[2][2])$
 $d[3][3] = \min(d[3][3], d[3][2] + d[2][3])$
 $d[3][4] = \min(d[3][4], d[3][2] + d[2][4])$

$d[4][1] = \min(d[4][1], d[4][2] + d[2][1])$
 $d[4][2] = \min(d[4][2], d[4][2] + d[2][2])$
 $d[4][3] = \min(d[4][3], d[4][2] + d[2][3])$
 $d[4][4] = \min(d[4][4], d[4][2] + d[2][4])$



	1	2	3	4
1	0	7	8	4
2	∞	0	1	-3
3	∞	∞	0	-4
4	∞	2	3	-1

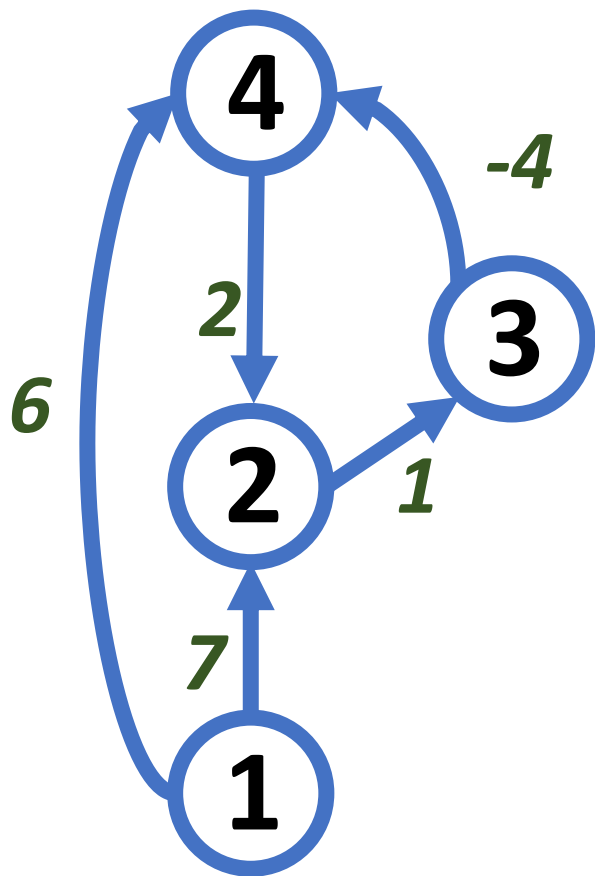
$k = 3$

$d[1][1] = \min(d[1][1], d[1][3] + d[3][1])$
 $d[1][2] = \min(d[1][2], d[1][3] + d[3][2])$
 $d[1][3] = \min(d[1][3], d[1][3] + d[3][3])$
 $d[1][4] = \min(d[1][4], d[1][3] + d[3][4])$

$d[2][1] = \min(d[2][1], d[2][3] + d[3][1])$
 $d[2][2] = \min(d[2][2], d[2][3] + d[3][2])$
 $d[2][3] = \min(d[2][3], d[2][3] + d[3][3])$
 $d[2][4] = \min(d[2][4], d[2][3] + d[3][4])$

$d[3][1] = \min(d[3][1], d[3][3] + d[3][1])$
 $d[3][2] = \min(d[3][2], d[3][3] + d[3][2])$
 $d[3][3] = \min(d[3][3], d[3][3] + d[3][3])$
 $d[3][4] = \min(d[3][4], d[3][3] + d[3][4])$

$d[4][1] = \min(d[4][1], d[4][3] + d[3][1])$
 $d[4][2] = \min(d[4][2], d[4][3] + d[3][2])$
 $d[4][3] = \min(d[4][3], d[4][3] + d[3][3])$
 $d[4][4] = \min(d[4][4], d[4][3] + d[3][4])$



	1	2	3	4
1	0	6	7	3
2	∞	-1	0	-4
3	∞	-2	-1	-5
4	∞	1	2	-2

$$k = 4$$

$$\begin{aligned} d[1][1] &= \min(d[1][1], d[1][4] + d[4][1]) \\ d[1][2] &= \min(d[1][2], d[1][4] + d[4][2]) \\ d[1][3] &= \min(d[1][3], d[1][4] + d[4][3]) \\ d[1][4] &= \min(d[1][4], d[1][4] + d[4][4]) \end{aligned}$$

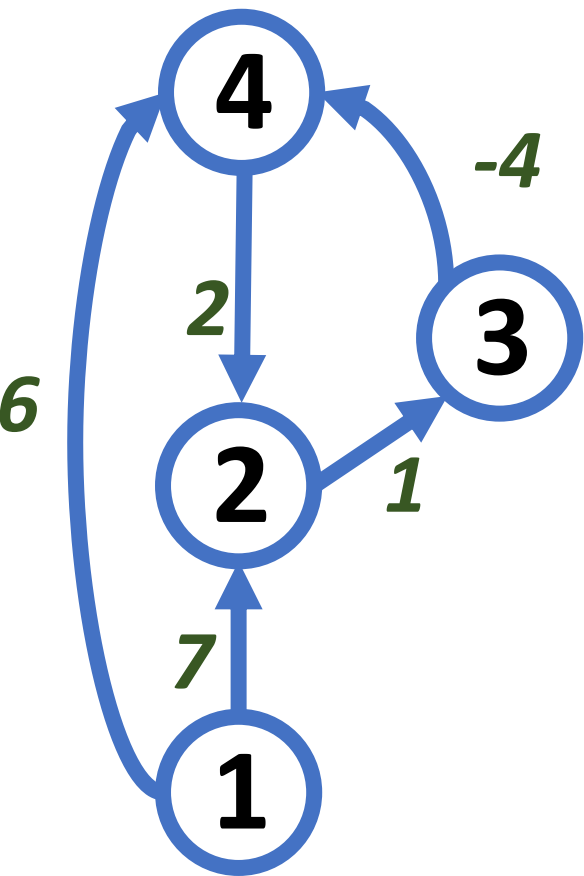
$$\begin{aligned} d[2][1] &= \min(d[2][1], d[2][4] + d[4][1]) \\ d[2][2] &= \min(d[2][2], d[2][4] + d[4][2]) \\ d[2][3] &= \min(d[2][3], d[2][4] + d[4][3]) \\ d[2][4] &= \min(d[2][4], d[2][4] + d[4][4]) \end{aligned}$$

$$\begin{aligned} d[3][1] &= \min(d[3][1], d[3][4] + d[4][1]) \\ d[3][2] &= \min(d[3][2], d[3][4] + d[4][2]) \\ d[3][3] &= \min(d[3][3], d[3][4] + d[4][3]) \\ d[3][4] &= \min(d[3][4], d[3][4] + d[4][4]) \end{aligned}$$

$$\begin{aligned} d[4][1] &= \min(d[4][1], d[4][4] + d[4][1]) \\ d[4][2] &= \min(d[4][2], d[4][4] + d[4][2]) \\ d[4][3] &= \min(d[4][3], d[4][4] + d[4][3]) \\ d[4][4] &= \min(d[4][4], d[4][4] + d[4][4]) \end{aligned}$$

Для этого можно сделать, например, следующий **критерий** "не существования пути". Итак, пусть на данном графе отработал обычный алгоритм Флойда. Тогда между вершинами i и j не существует кратчайшего пути тогда и только тогда, когда найдётся такая вершина t , достижимая из i и из которой достижима j , для которой выполняется $d[t][t] < 0$.

Кроме того, при использовании алгоритма Флойда для графов с отрицательными циклами следует помнить, что возникающие в процессе работы расстояния могут сильно уходить в минус, экспоненциально с каждой фазой. Поэтому следует принять меры против целочисленного переполнения, ограничив все расстояния снизу какой-нибудь величиной (например, $-\text{INF}$).



	1	2	3	4
1	0	6	7	3
2	∞	-1	0	-4
3	∞	-2	-1	-5
4	∞	1	2	-2

$k = 4$

$d[1][1] = \min(d[1][1], d[1][4] + d[4][1])$
 $d[1][2] = \min(d[1][2], d[1][4] + d[4][2])$
 $d[1][3] = \min(d[1][3], d[1][4] + d[4][3])$
 $d[1][4] = \min(d[1][4], d[1][4] + d[4][4])$

$d[2][1] = \min(d[2][1], d[2][4] + d[4][1])$
 $d[2][2] = \min(d[2][2], d[2][4] + d[4][2])$
 $d[2][3] = \min(d[2][3], d[2][4] + d[4][3])$
 $d[2][4] = \min(d[2][4], d[2][4] + d[4][4])$

$d[3][1] = \min(d[3][1], d[3][4] + d[4][1])$
 $d[3][2] = \min(d[3][2], d[3][4] + d[4][2])$
 $d[3][3] = \min(d[3][3], d[3][4] + d[4][3])$
 $d[3][4] = \min(d[3][4], d[3][4] + d[4][4])$

$d[4][1] = \min(d[4][1], d[4][4] + d[4][1])$
 $d[4][2] = \min(d[4][2], d[4][4] + d[4][2])$
 $d[4][3] = \min(d[4][3], d[4][4] + d[4][3])$
 $d[4][4] = \min(d[4][4], d[4][4] + d[4][4])$

Когда использовать эти алгоритмы?

Что нужно найти?	Плотный граф $m = O(n^2)$	Разреженный граф $m = O(n)$
Кратчайшее расстояние от одной вершины до остальных	Дейкстра на базе массива $T = O(n^2 + m) = O(n^2)$	Дейкстра на базе бинарной кучи $T = O(n \log n + m \log n) = O(n \log n)$
Кратчайшие расстояния между всеми парами вершин	Фloyd-Уоршелл $T = O(n^3)$	Дейкстра на базе бинарной кучи $T = O(n^* n \log n) = O(n^2 \log n)$

Алгоритм Беллмана-Форда

- Алгоритм поиска кратчайшего пути во взвешенном графе. В отличие от алгоритма Дейкстры, алгоритм Беллмана — Форда допускает рёбра с отрицательным весом.
- Важная особенность: после выполнения k фаз алгоритм Форда-Беллмана корректно находит все кратчайшие пути, длина которых (по числу рёбер) не превосходит k .

Алгоритм Беллмана-Форда

1. Вершине-входу присваиваем потенциал $\varphi_1 = 0$; остальным вершинам – потенциал $\varphi_i = \infty$, $i = 1 \dots n$, где n - число вершин в рассматриваемом графе.
2. Для каждой вершины i , смежной с вершиной j из массива вершин L , находим $\varphi_j + l(j, i)$ и проверяем неравенство

$$\varphi_j + l(j, i) < \varphi_i.$$

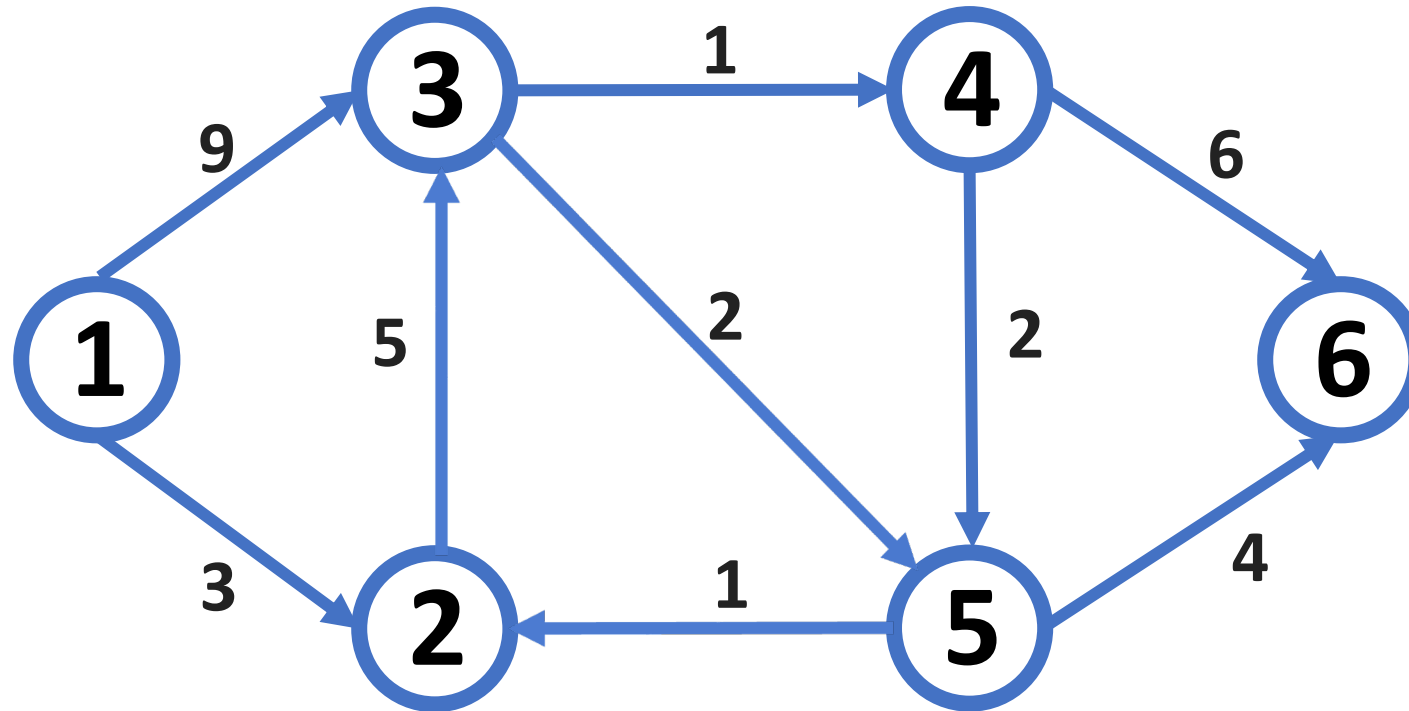
Если неравенство выполняется, то

$$\varphi_i := \varphi_j + l(j, i).$$

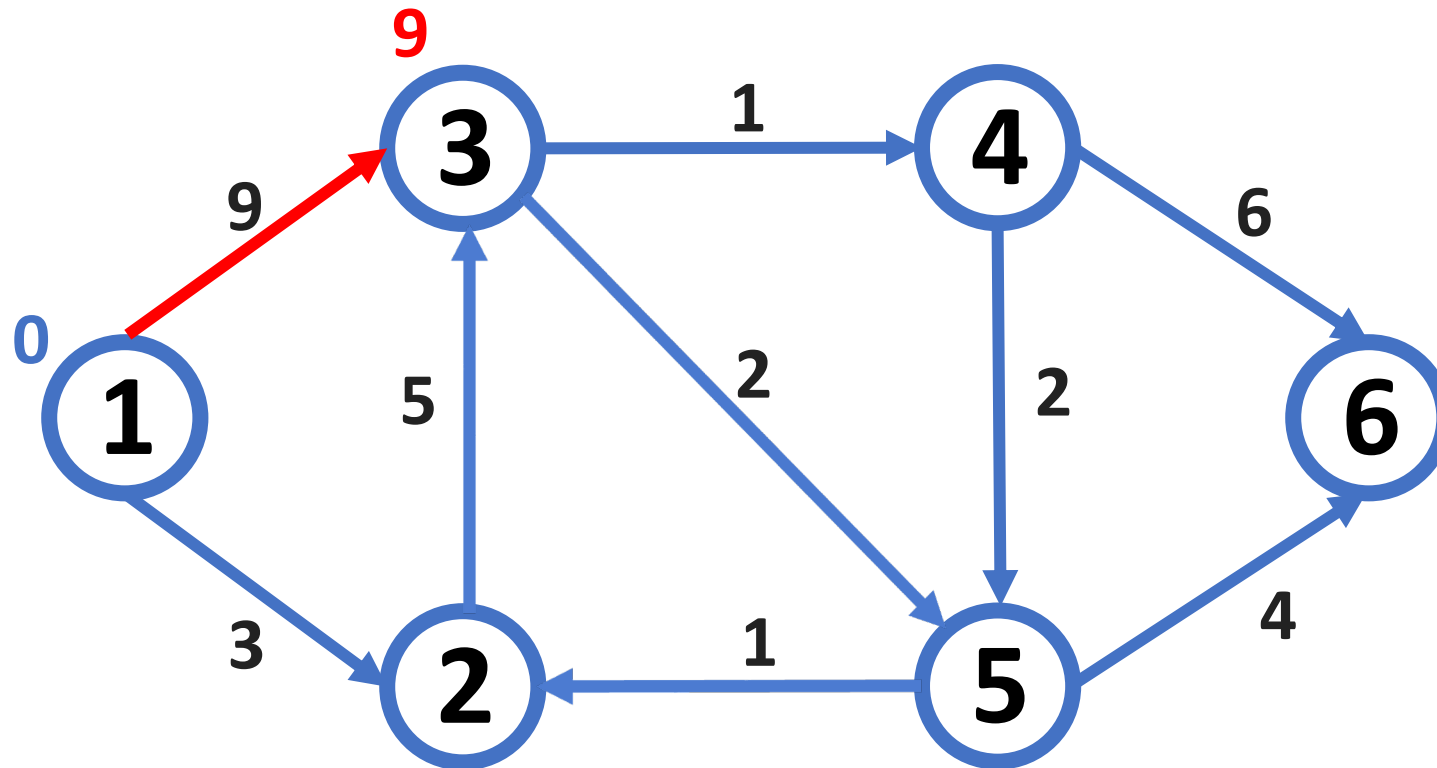
Вершину i заносим в массив L_1 .

3. Проверяем массив $L_1 \neq \emptyset$. Если неравенство выполняется $L := L_1$, осуществляем переход к п. 2.
4. Конец алгоритма нахождения кратчайших путей.

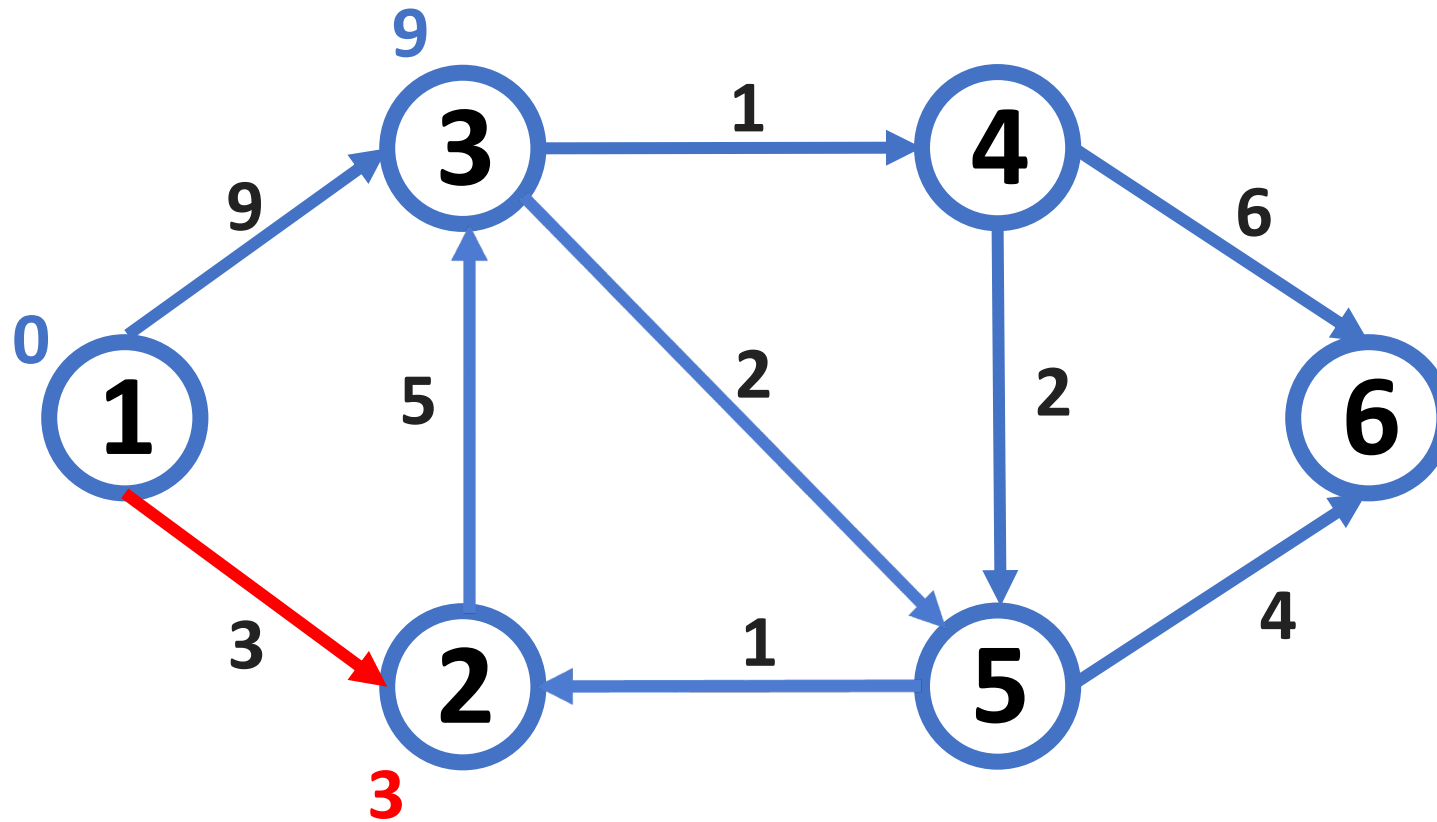
Найти кратчайшие пути из 1 вершины в остальные алгоритмом Беллмана-Форда



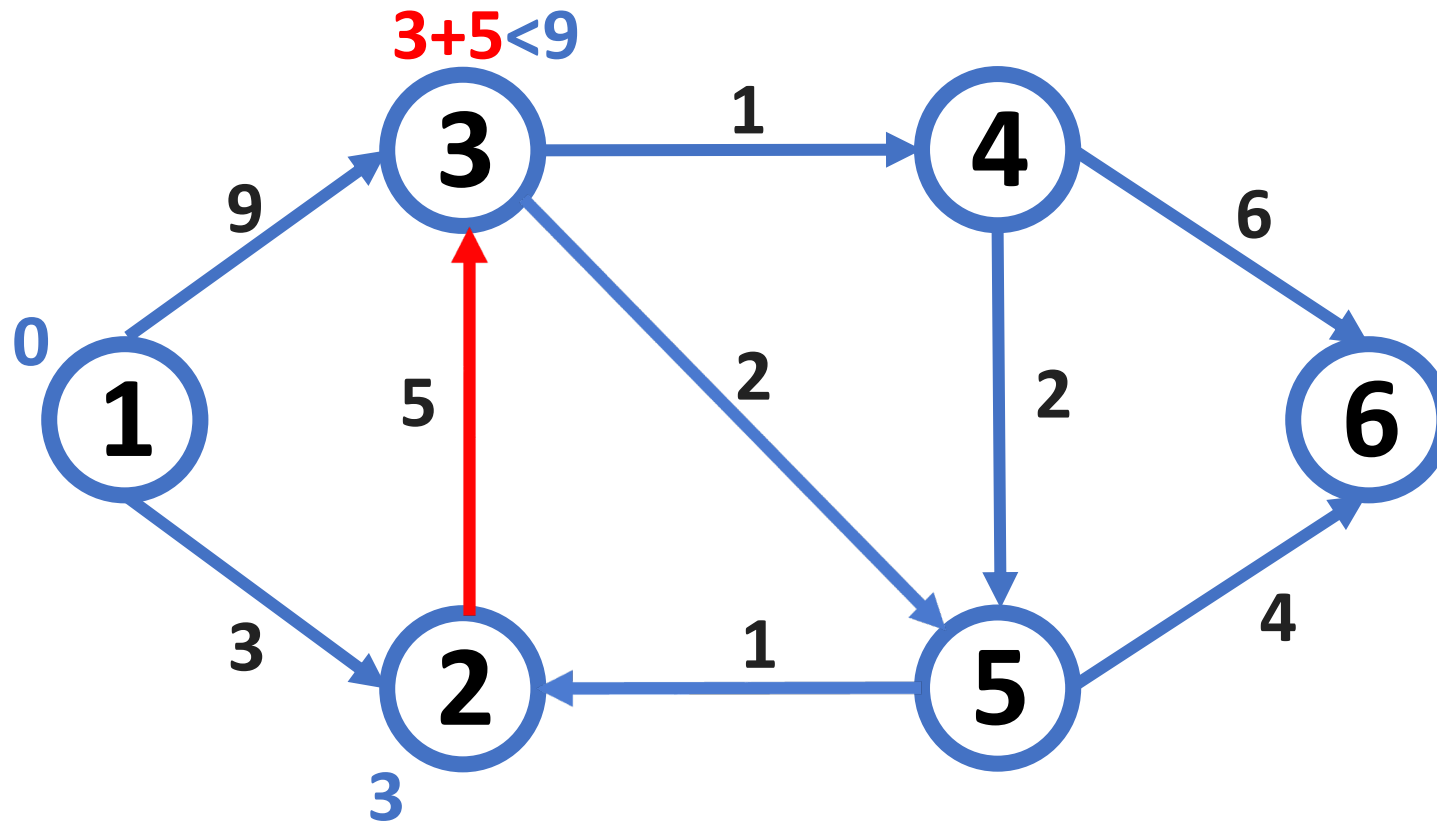
1 итерация



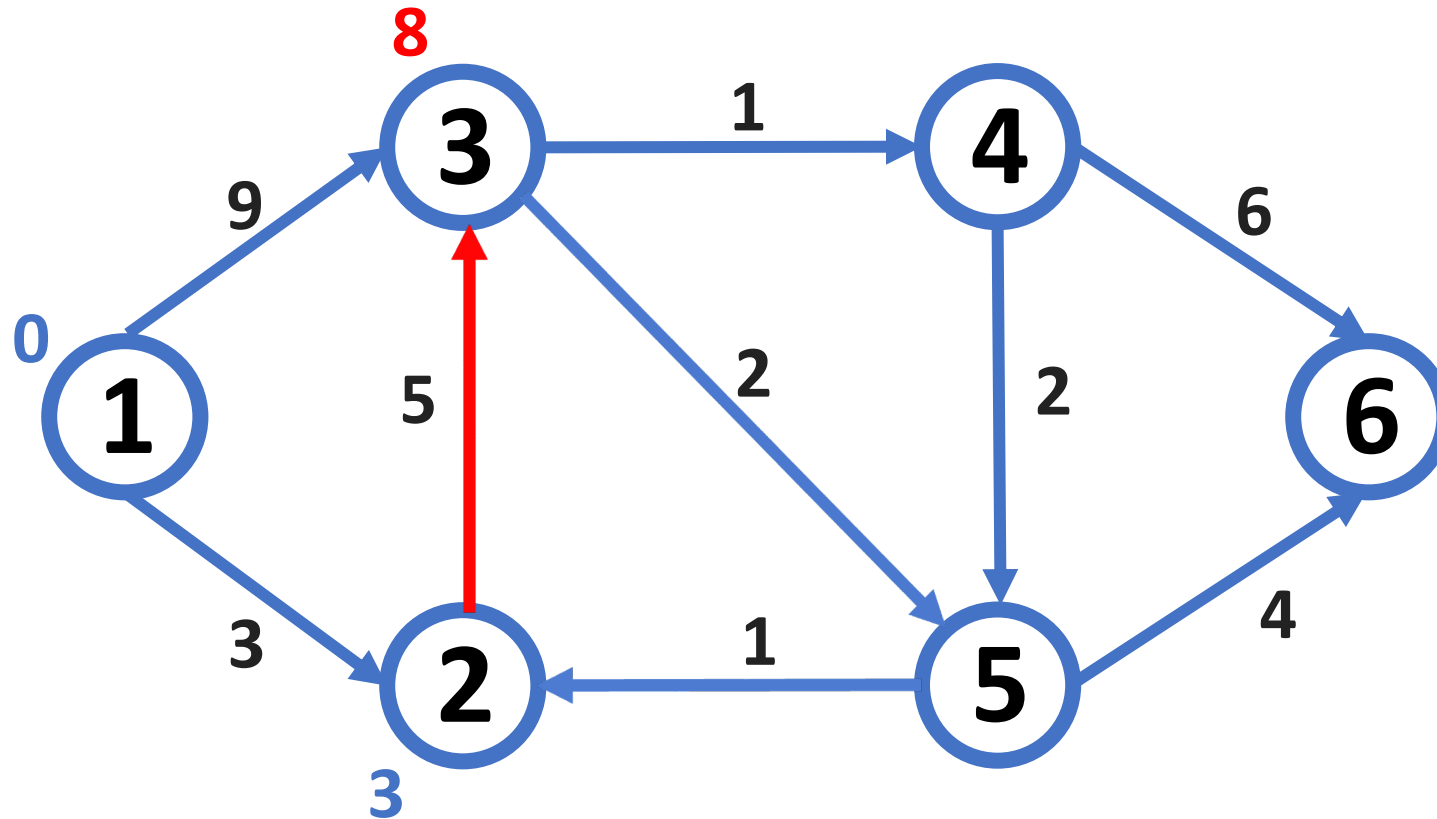
1 итерация



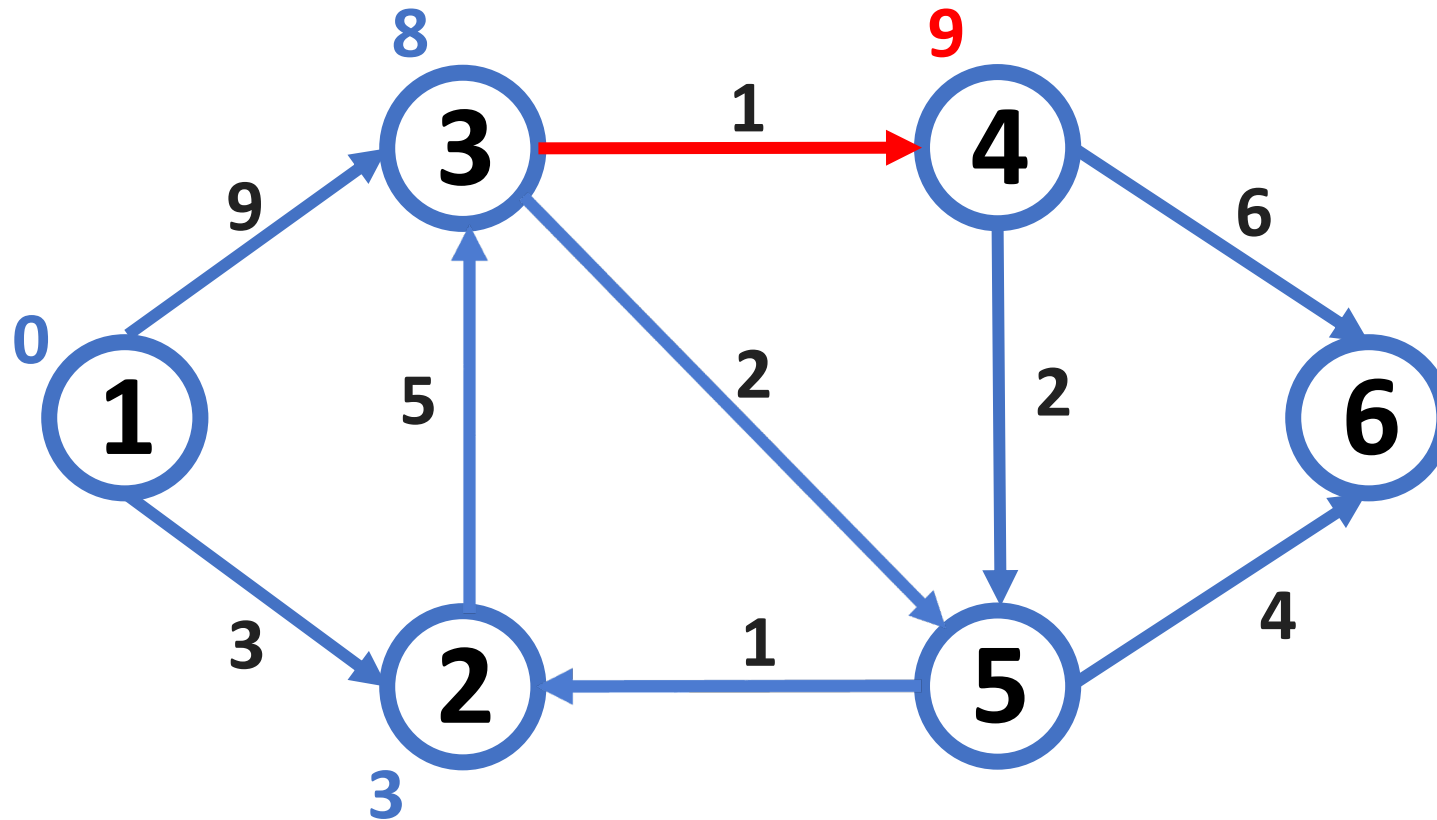
1 итерация



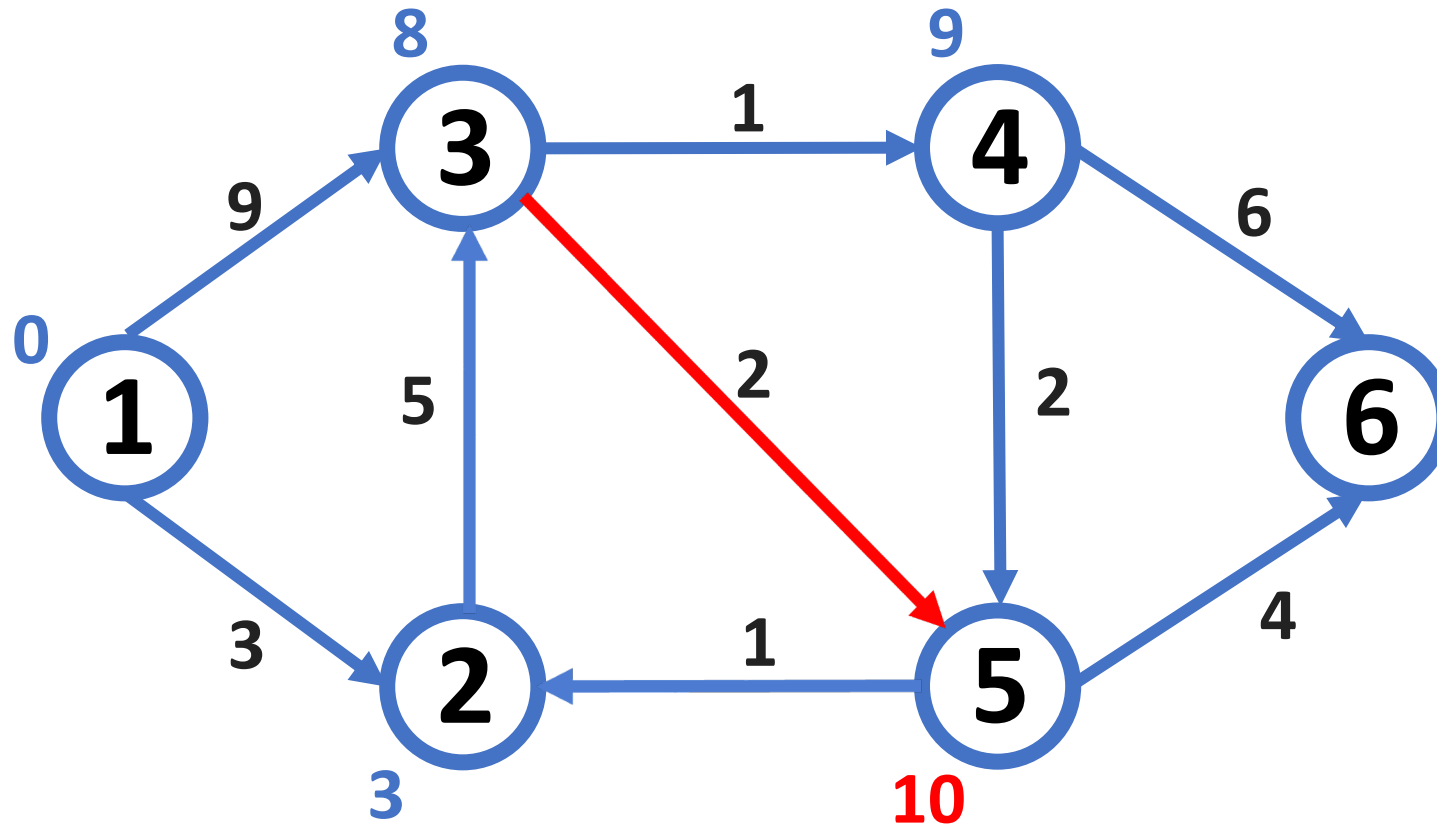
1 итерация



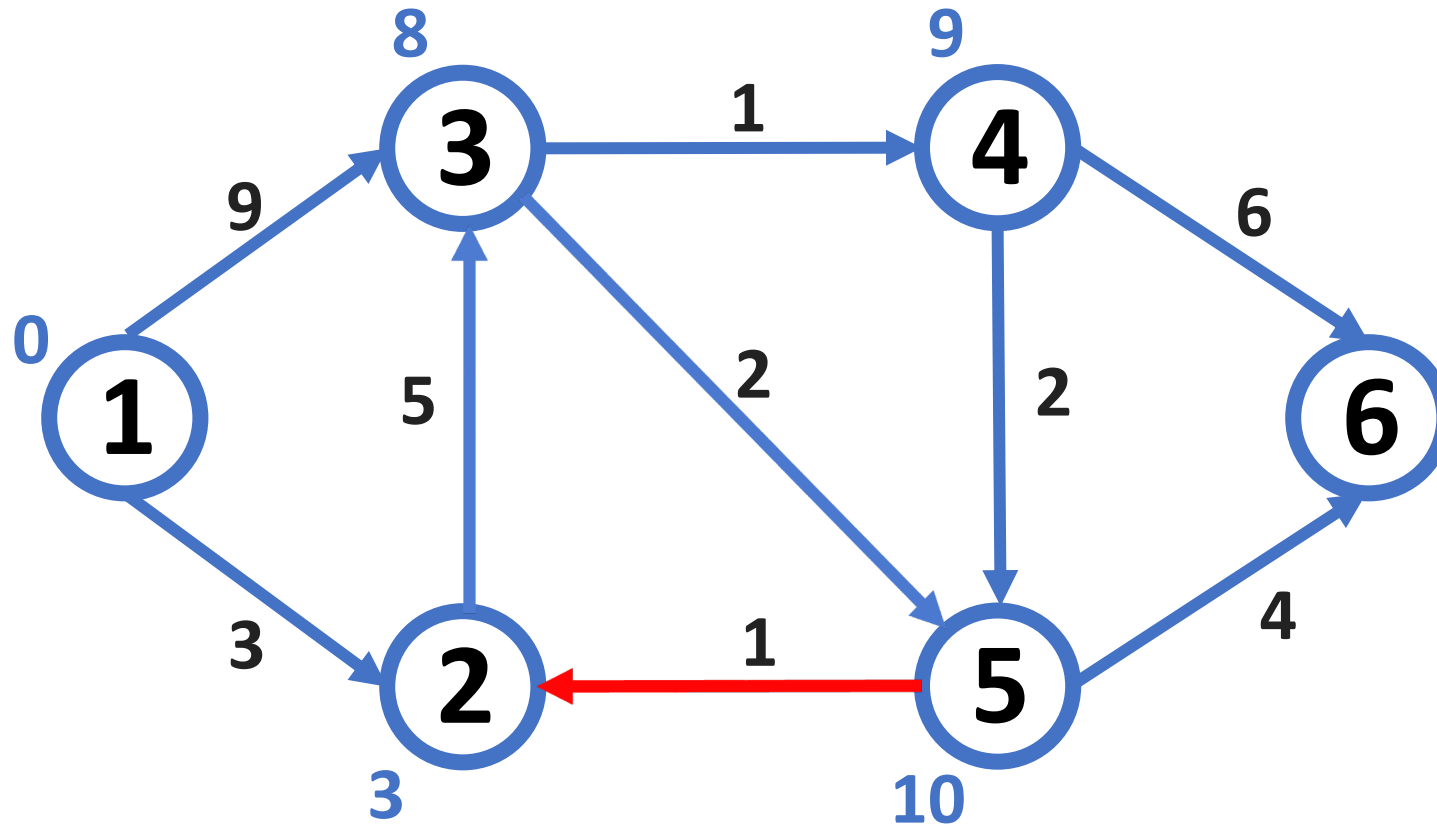
1 итерация



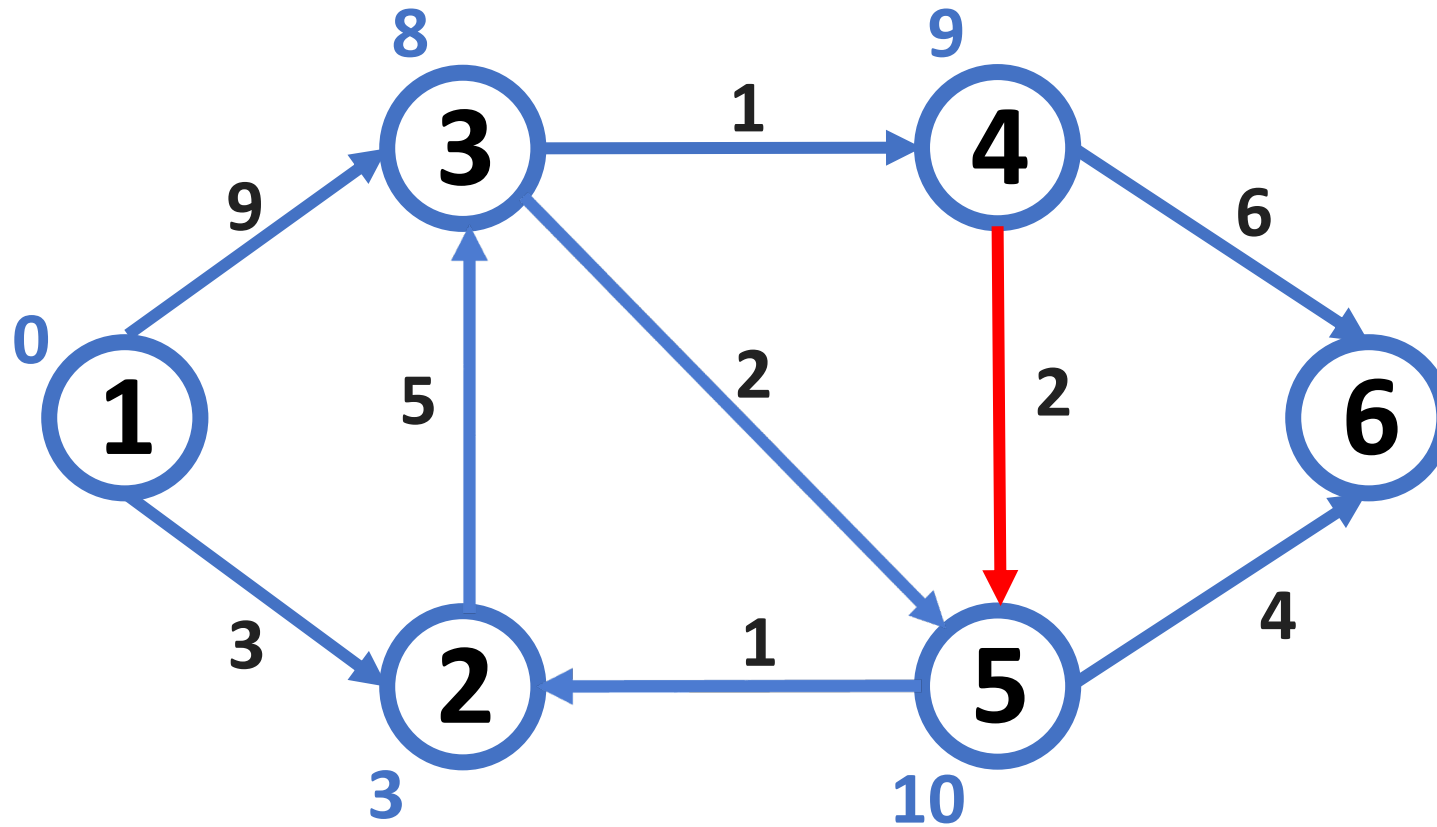
1 итерация



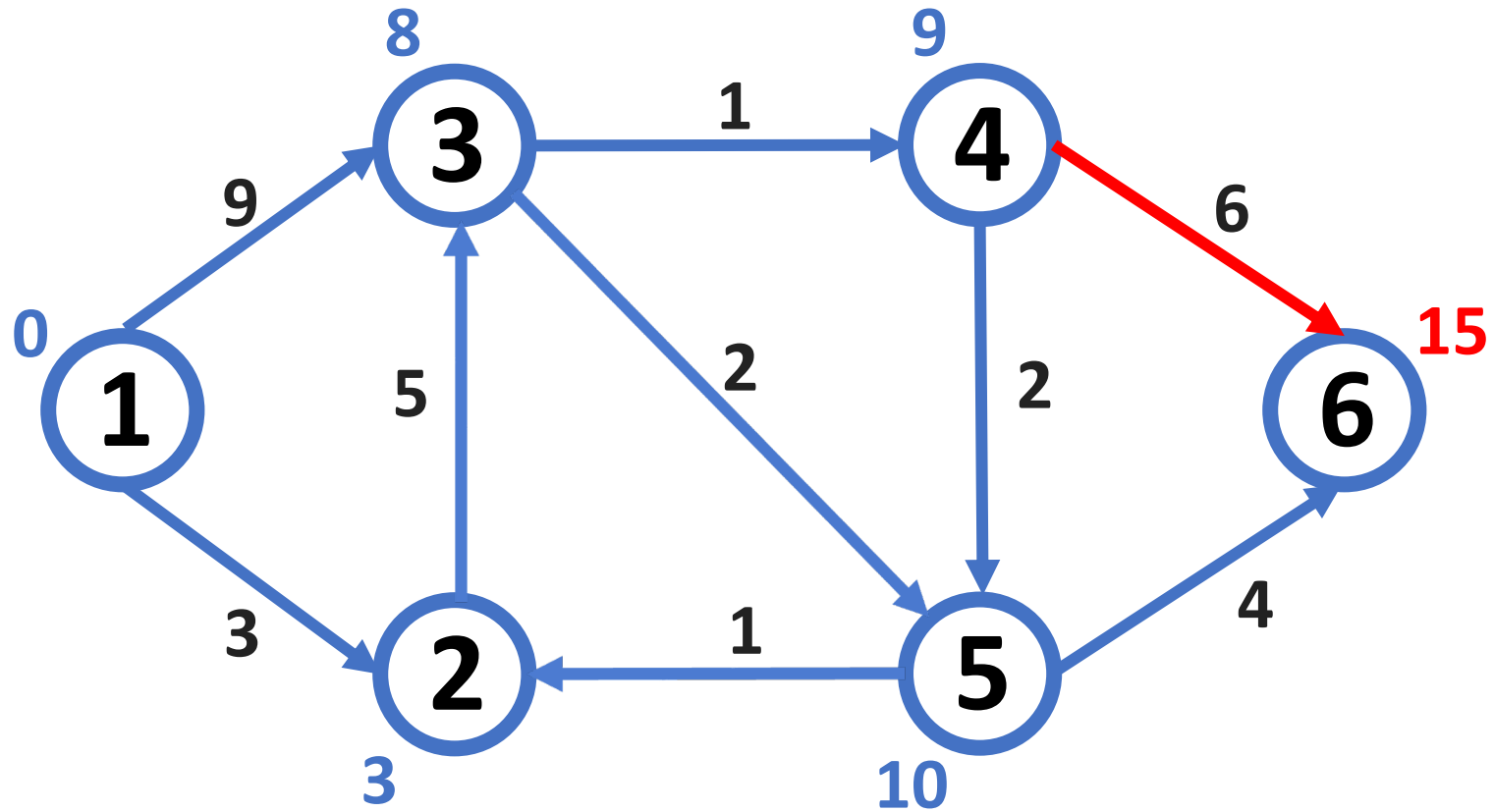
1 итерация



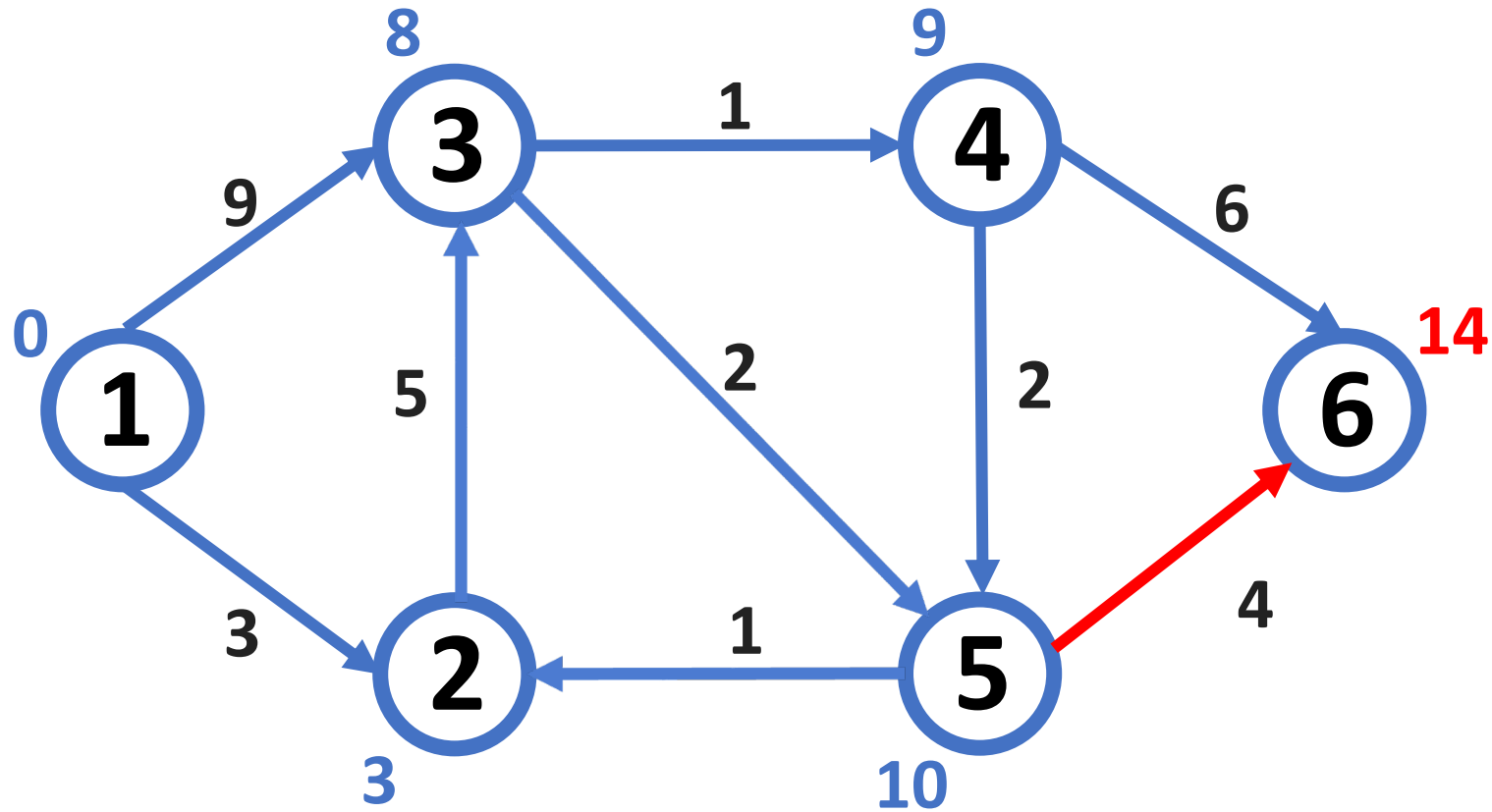
1 итерация



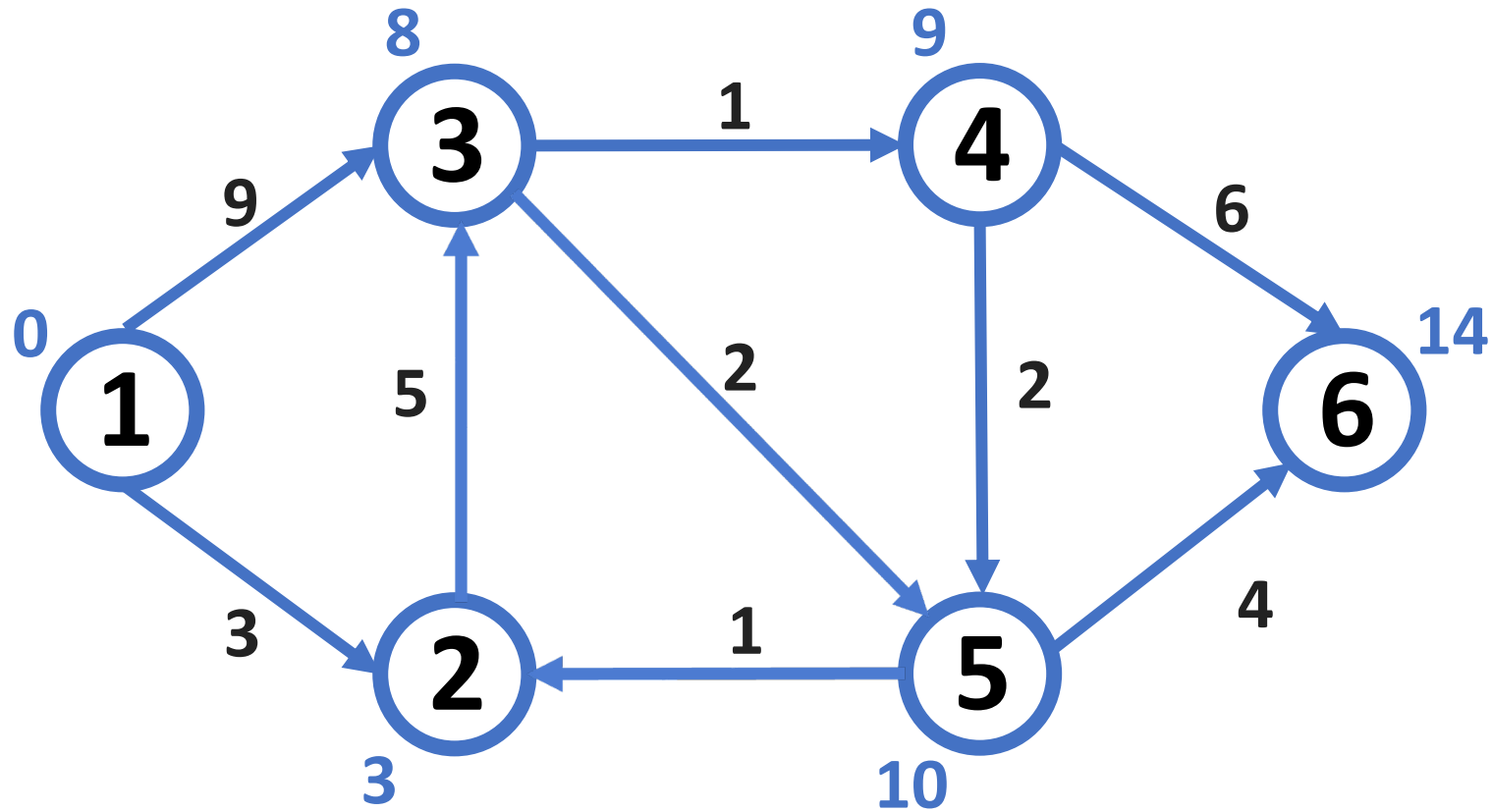
1 итерация



1 итерация



2 итерация – больше релаксаций не найдено, это редкость



Простая реализация

```
struct edge {
    int a, b, cost;
};

int n, m, v;
vector<edge> e;
const int INF = 1000000000;

void solve() {
    vector<int> d (n, INF);
    d[v] = 0;
    for (int i=0; i<n-1; ++i)
        for (int j=0; j<m; ++j)
            if (d[e[j].a] < INF)
                d[e[j].b] = min (d[e[j].b], d[e[j].a] + e[j].cost);

    // вывод d, например, на экран
}
```

Проверка "if (d[e[j].a] < INF)" нужна, только если граф содержит рёбра отрицательного веса: без такой проверки бы происходили релаксации из вершин, до которых пути ещё не нашли, и появлялись бы некорректные расстояния вида $\infty - 1$, $\infty - 2$, и т.д.

Улучшенная реализация

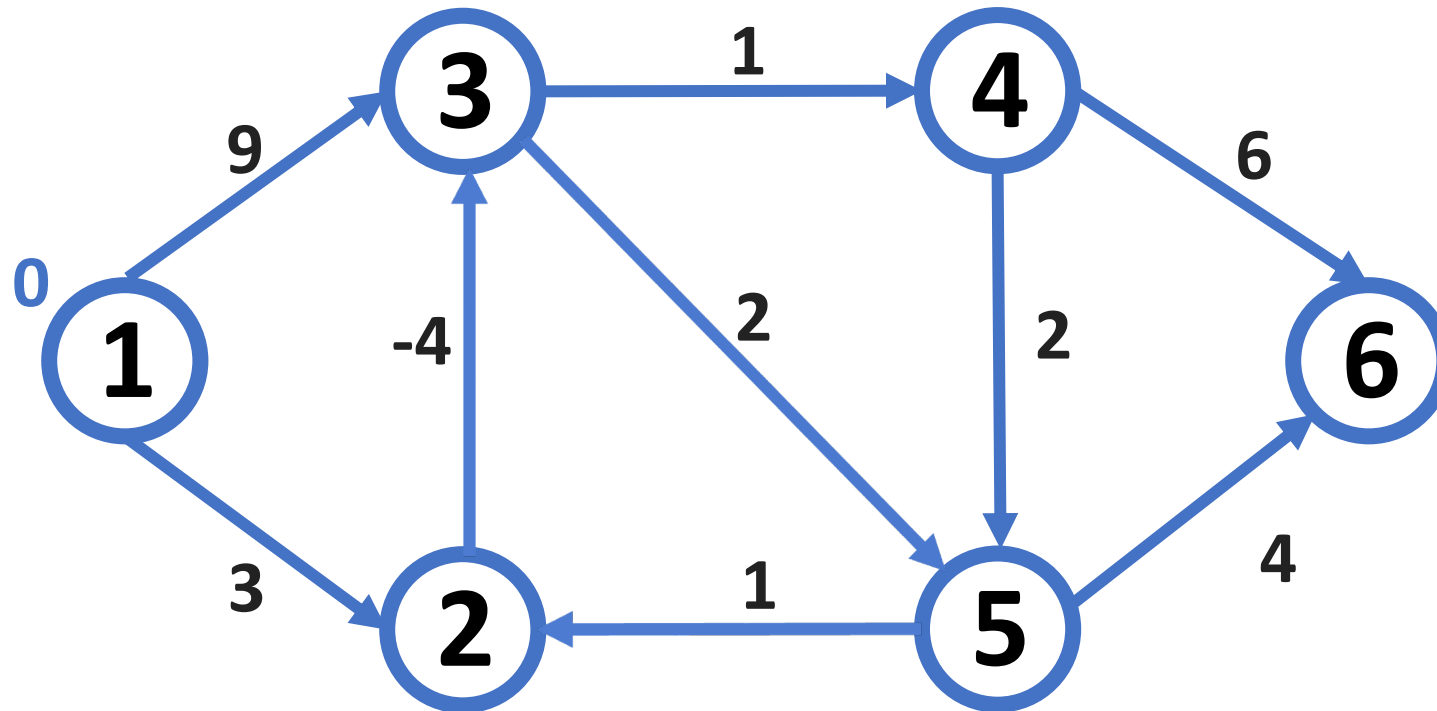
Этот алгоритм можно несколько ускорить: зачастую ответ находится уже за несколько фаз, а оставшиеся фазы никакой полезной работы не происходит, лишь впустую просматриваются все рёбра. Поэтому будем хранить флаг того, изменилось что-то на текущей фазе или нет, и если на какой-то фазе ничего не произошло, то алгоритм можно останавливать. (Эта оптимизация не улучшает асимптотику, т.е. на некоторых графах по-прежнему будут нужны все $n - 1$ фаза, но значительно ускоряет поведение алгоритма "в среднем", т.е. на случайных графах.)

С такой оптимизацией становится вообще ненужным ограничивать вручную число фаз алгоритма числом $n - 1$ — он сам остановится через нужное число фаз.

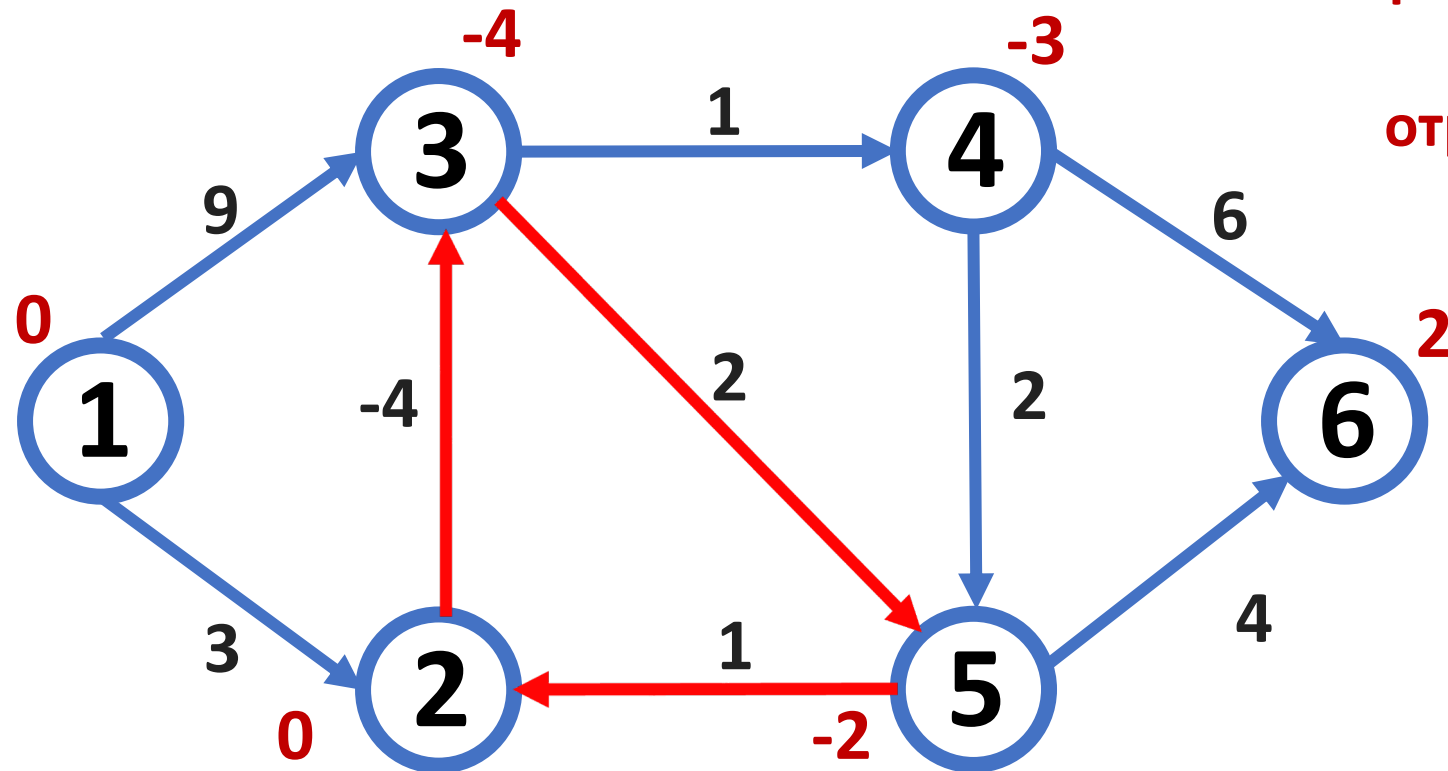
```
void solve() {
    vector<int> d (n, INF);
    d[v] = 0;
    for (;;) {
        bool any = false;
        for (int j=0; j<m; ++j)
            if (d[e[j].a] < INF)
                if (d[e[j].b] > d[e[j].a] + e[j].cost) {
                    d[e[j].b] = d[e[j].a] + e[j].cost;
                    any = true;
                }
        if (!any) break;
    }
    // вывод d, например, на экран
}
```

Другой граф

1 итерация



Другой граф
n итерация



На n-ой итерации
тоже получилось
провести
релаксацию, значит,
есть цикл
отрицательного веса

Нетрудно понять, что алгоритм Форда-Беллмана сможет **бесконечно делать релаксации** среди всех вершин этого цикла и вершин, достижимых из него. Следовательно, если не ограничивать число фаз числом $n - 1$, то алгоритм будет работать бесконечно, постоянно улучшая расстояния до этих вершин.

Отсюда мы получаем **критерий наличия достижимого цикла отрицательного веса**: если после $n - 1$ фазы мы выполним ещё одну фазу, и на ней произойдёт хотя бы одна релаксация, то граф содержит цикл отрицательного веса, достижимый из v ; в противном случае, такого цикла нет.

```
void solve() {
    vector<int> d (n, INF);
    d[v] = 0;
    vector<int> p (n, -1);
    int x;
    for (int i=0; i<n; ++i) {
        x = -1;
        for (int j=0; j<m; ++j)
            if (d[e[j].a] < INF)
                if (d[e[j].b] > d[e[j].a] + e[j].cost) {
                    d[e[j].b] = max (-INF, d[e[j].a] + e[j].cost);
                    p[e[j].b] = e[j].a;
                    x = e[j].b;
                }
    }

    if (x == -1)
        cout << "No negative cycle from " << v;
```

Обобщение

Что нужно найти?	Есть цикл отрицательного веса?	Плотный граф $m = O(n^2)$	Разреженный граф $m = O(n)$
Кратчайшее расстояние от одной вершины до остальных	Да	Беллман-Форд $O(n m) = O(n^3)$	Беллман-Форд $O(nm) = O(n^2)$
	Нет	Дейкстра на базе массива $O(n^2 + m) = O(n^2)$	Дейкстра на базе бинарной кучи $O(n \log n + m \log n) = O(n \log n)$
Кратчайшие расстояния между всеми парами вершин	Да	Фloyd-Уоршелл $O(n^3)$	Беллман-Форд $O(n * nm) = O(n^3)$
	Нет	Фloyd-Уоршелл $O(n^3)$	Дейкстра на базе бинарной кучи $O(n * n \log n) = O(n^2 \log n)$

КОНТАКТЫ



LinkedIn

<https://ru.linkedin.com/in/mariia-gordenko-78617618b>



Telegram, Instagram
oduvan_ja



Адрес электронной почты

mgordenko@hse.ru mkgordenko@gmail.com



Личная страница

<https://www.hse.ru/staff/gordenko>